Enzyme Constrained Thermodynamic Flux Layer

Contents

1 2026.09.01 - Build, reconstruction, and first diagnostic runs ✗ 2
- 1.1 Why a flux layer at all, stated as a cost argument ✗ . . . 2
- 1.2 The relaxation, term by term ✗ . . . 2
- 1.3 Thermodynamics: free versus anchored, and why the distinction is the experiment ✗ . . . 3
- 1.4 The exactness budget, and why it is a fifth arm rather than a footnote ✗ . . . 4
- 1.5 The reconstruction: what the layer actually rests on ✗ . . . 4
- 1.6 Closing the deferred metabolite-identifier gap ✗ . . . 6
- 1.7 Porting to another organism, and why kinetics is the only real barrier ✗ . . . 7
- 1.8 Always emit fluxes, or sample on demand? One switch, and it is not architectural ✗ . . . 7
- 1.9 The diagnostic experiment ✗ . . . 9
- 1.10 The sharpest result so far: a sigmoid box cannot reach a sparse flux vector ✗ . . . 10
- 1.11 Media: the FBA layer works, the ontology objects do not reach it yet ✗ . . . 11
- 1.12 Literature captured for this work ✗ . . . 12
- 1.13 Reproducing everything ✗ . . . 13
- 1.14 Results: five arms, three seeds, 20 epochs ✗ . . . 13
- 1.15 Amortized flux sampling, and what the width comparison does and does not show ✗ . . . 15
- 1.16 Next changes, in the order they should happen ✗ . . . 15
- 1.17 What is not claimed ✗ . . . 16

2 2026.09.03 - The arm comparison against a calibrated null, and the kinetics inputs ✗ 16
- 2.1 Why this was run instead of a longer budget ✗ . . . 16
- 2.2 The null, measured rather than assumed ✗ . . . 16
- 2.3 No arm is distinguishable from that null ✗ . . . 16
- 2.4 Regularization does not rescue it ✗ . . . 17
- 2.5 What is not claimed ✗ . . . 17
- 2.6 Kinetic parameter inputs, resolved ✗ . . . 17
- 2.7 One correction to the sections above ✗ . . . 18

3 2026.09.08 - Gibbs dissipation is a readout, not a constraint ✗ 18

4 2026.09.08 - The diagram source is now a .drawio.png, exported at true Nature size ✗ 19

5 2026.09.09 - The protein node: a predicted abundance replaces the nominal E_bar ✗ 19

6 2026.09.10 - Predicted parameters wired, and the arms measured ✗ 21
- 6.1 The predicted k_cat and K_M tier ✗ . . . 21
- 6.2 The merged reaction energy table ✗ . . . 22
- 6.3 The constraint warmup ✗ . . . 22
- 6.4 The four cells ✗ . . . 22
- 6.5 The head starts seven sigmas off the target, and that is the whole epoch-0 gap ✗ . . . 23
- 6.6 The shipped delta-G tables are kcal/mol, and the union was mixing units ✗ . . . 23
- 6.7 The head normalization closes the data-term gap, measured in training ✗ . . . 24

7 2026.09.10 - Shipped energies are kcal/mol, lipids are priced, transport is computed ✗ 24
- 7.1 1. yeast-GEM 9's shipped delta-G columns are kcal/mol, labeled kJ/mol ✗ . . . 24
- 7.2 2. The transport arm of the Figure 3 cross closes; the chemistry cloud does not ✗ . . . 25
- 7.3 3. The transport term is computed rather than recovered ✗ . . . 25
- 7.4 4. The tier-b lipid structures are written and priced ✗ . . . 25
- 7.5 The wiring, and what is still running ✗ . . . 26
- 7.6 What was rewritten, and where ✗ . . . 26
- 7.7 One number corrected in place ✗ . . . 26
- 7.8 The head fix corrects the loss and appears to cost the escape ✗ . . . 26
- 7.9 The constraint terms are the blocker, and the head offset was a second, independent defect ✗ . . . 27
- 7.10 The endpoint, on the four cells that finished ✗ . . . 27
- 7.11 The constraints destroy an escaped solution, at a weight below a twentieth of the data term ✗ . . . 28
- 7.12 Final wave-5 table ✗ . . . 28
- 7.13 G9 localizes the blocking terms: it is mass balance and the box, not the module ✗ . . . 29

✓read and checked ■read, provisional ✗agent-written, not yet read

1 2026.09.01 - Build, reconstruction, and first diagnostic runs ✗

Branch `feat/metabolism-flux-module`. The model specification this implements is *CGT-Metabolism Flux Layer -- model math and the Flux Cone Learning comparison* (`plan.cgt-metabolism-flux-layer.2026.07.26`), its derivations are in *CGT Metabolism Flux Layer Explainer* (`plan.cgt-metabolism-flux-layer.explainer`), and the Track A run log and noise ceilings are in *CGT-Metabolism -- pigment prediction, metabolome transfer, and the enzyme-constrained model class* (`plan.cgt-metabolism.2026.07.25`).

1.1 Why a flux layer at all, stated as a cost argument ✗

The motivating suspicion is that the multimodal Cell Graph Transformer under-performs on production phenotypes because it carries no representation of metabolism. Betaxanthin sits at Pearson 0.128 against a replicate-based noise ceiling of 0.914, which is 14 % of what the measurement supports.

Constraint-based modeling already answers "which flux vectors are possible", so the question is why not simply use it. Two reasons, and both are about cost rather than accuracy.

A solver's cost is per genotype and does not amortize. Merzbacher et al. (2025) sample one flux cone per single deletion by Markov-chain Monte Carlo: 1,159 yeast deletions, 124 samples each, 4.43 GB. The pairwise extension is $\binom{1161}{2} = 673{,}380$ independent random walks, roughly 2.6 TB, and their own Methods call the walk "computationally costly because it requires running a random walk on a high-dimensional flux space that needs to reach mixing time." An amortized model pays that cost once at training time, and a double deletion is the same forward pass as a single.

A program cannot learn. A linear program has no use for a phenotype measurement. Adding thermodynamics makes it worse rather than better: Smith et al. (2026), the *Thermo-Flux* protocol, enforces the second law as $\Delta_r G_j \nu_j < 0$, which is bilinear, and implements it with **one binary indicator per reaction** under a big-M pair. The result is a mixed-integer program whose cost, in the authors' words, grows because "the number of possible direction combinations scales exponentially with the number of reactions", with a 24 h wall-time cap per model on an HPC cluster.

So the design target is a layer that keeps the physics and drops the integers.

1.2 The relaxation, term by term ✗

Every hard constraint is either exact by construction or a smooth penalty. No binary variables anywhere.

Table 1. Every hard constraint, and how it is enforced here versus in a solver. Thermo-Flux (Smith et al. 2026) realizes the second law with one binary indicator per reaction under a big-M pair, making the program mixed-integer. Two constraints are exact by construction here and cost nothing in the loss; the other four are smooth penalties and none needs an integer variable.

constraint	Thermo-Flux / GECKO form	here
box and directionality	$lb_j \le \nu_j \le ub_j$	**exact**, sigmoid
enzyme capacity	$\nu_j \le k_{\text{cat}} E$	**exact**, folded into the box
mass balance	$S\nu = 0$	soft, scale-relative
second law	$\Delta_r G_j \nu_j < 0$ + one binary per reaction	soft hinge, no binary
Gibbs dissipation	$g_{\text{diss}} \le g_{\text{lim}}$	soft hinge
protein budget	$\sum_g \text{MW}_g E_g \le P_{\text{avail}}$	soft hinge

Availability chain. A deletion enters as a hard zero, not as a learned gate:

$$\gamma_g = \begin{cases} 0, & g \text{ deleted in } p \\ \sigma(w^\top h_g^{\text{g}}), & \text{otherwise} \end{cases} \qquad c_u = -\tfrac{1}{\beta} \log \sum_{g \in C_u} e^{-\beta \gamma_g}, \qquad c_j = \min\Big(1, \sum_{u:\rho(u)=j} c_u\Big)$$

A complex takes a softmin over its subunits, since the scarcest one limits it; isozymes sum, since their capacities add. A reaction with no gene-protein-reaction rule keeps $c_j = 1$: an unannotated reaction is one nobody has assigned a gene to, not one no gene catalyzes, and zeroing it would let a missing annotation delete a reaction.

The dynamic box. Availability and capacity scale the bounds, and the flux is written inside them:

$$\bar{v}_j^u = \min\Big(v_j^u,\ c_j \sum_{u:\rho(u)=j} k_{\text{cat},u} \bar{E}\Big), \qquad v_j = \bar{v}_j^\ell + (\bar{v}_j^u - \bar{v}_j^\ell)\, \sigma(z_j)$$

One line does four jobs for free. It enforces the box exactly; it enforces directionality for the 2,463 irreversible reactions; it makes a deletion collapse the box to the single point 0, so the deletion bites through the parameterization rather than through a penalty the data term could trade against; and it therefore contributes **no term to the loss**.

Mass balance, scale-relative and rank-restricted.

$$\omega_i(v) = \tfrac{1}{2}\sum_j |S_{ij}v_j|, \qquad C_{\text{bal}} = \frac{1}{|\mathcal{M}^\dagger|}\sum_{i\in\mathcal{M}^\dagger}\left(\frac{[Sv]_i}{\text{sg}(\omega_i(v))+\epsilon}\right)^2$$

$\mathcal{M}^\dagger$ is the rank$(S) = 2{,}593$ independent rows; the other 213 are linear combinations whose residual is already determined, so penalizing them weights some directions of the error several times over. sg is a stop-gradient, without which the cheapest way to shrink the ratio is to carry no flux at all.

The second law, relaxed.

$$C_{\text{th}} = \frac{1}{|\mathcal{J}|}\sum_{j\in\mathcal{J}}\frac{\text{relu}\,(v_j\Delta_r G_j + \epsilon)}{|v_j||\Delta_r G_j| + \epsilon}$$

$\mathcal{J}$ excludes exchange reactions, the biomass reaction, and water transport, which are the exemptions Thermo-Flux's Box 2 specifies. Two details are not cosmetic. The ϵ is needed because the paper's own big-M pair admits $v_j = \Delta_r G_j = 0$ under either value of the binary, so the strict inequality of its Eq. (10) is never actually realized and a smooth version has to name its tolerance. The denominator makes the term a dimensionless fraction of the available driving force: measured at initialization on a real batch, the raw hinge is about 72 against a data loss of about 2, so an unnormalized term does not regularize the model, it replaces it.

Loop-freedom is free, and that is the whole trick. $\Delta_r G$ is a difference of potentials, so it sums to zero around any cycle and no cycle can run downhill everywhere. Requiring $v_j\Delta_j \le 0$ therefore forbids internal loops with no integer variables, which is exactly the property that makes loopless flux balance analysis a mixed-integer program.

Enzyme demand and the budget. Enzyme is a function of flux, never predicted:

$$E_g(v) = \sum_{j:\Pi_{gj}>0}\frac{\sqrt{v_j^2+\delta}}{k_{\text{cat},gj}}, \qquad P(v) = \sum_g \text{MW}_g E_g(v), \qquad C_{\text{bud}} = \text{relu}\Big(\tfrac{P(v)}{P_{\text{avail}}} - 1\Big)$$

One-sided, with no reward for approaching the budget. With 1,538 free internal directions, "use up your budget" is maximized by futile cycles, and enzyme-constrained flux balance analysis never rewards protein usage either.

Gibbs dissipation. Following Niebel et al. (2019) via Thermo-Flux,

$$g_{\text{diss}} = \sum_{j\in\text{exchange}} \Delta_r G_j\, v_j \;\le\; g_{\text{lim}}, \qquad g_{\text{lim}} = 3700\ \text{J gDW}^{-1}\,\text{h}^{-1}$$

implemented as $\text{relu}(g_{\text{diss}}/g_{\text{lim}} - 1)$. A squared excess divided by g_{lim}^2 evaluates to about 4×10^4 at initialization and its gradient drives the whole model to NaN within one step, which is what the first run did.

1.3 Thermodynamics: free versus anchored, and why the distinction is the experiment ✗

Following Thermo-Flux Eq. (4), the in-cell reaction energy decomposes into three terms:

$$\Delta_r G_j = \underbrace{\Delta_r G'^{\circ}_j}_{\text{tabulated}} + \underbrace{RT\sum_{i\neq \text{H}^+} S_{ij}\ln c_i}_{\text{concentration}} + \underbrace{\Delta_r G^t_j}_{\text{transport}}$$

Free mode learns an unconstrained potential μ_i and sets $\Delta_j = \sum_i S_{ij}\mu_i$. It needs no data and still kills loops, but μ is identified only up to an affine map, so it is not an energy and cannot be checked against anything.

Anchored mode takes $\Delta_r G'^{\circ}_j = \sum_i S_{ij}\Delta_f G'^{\circ}_i$ from the genome-scale model's own shipped table, learns $\ln c_i$ squashed into the physiological window [0.1 μM, 10 mM] that Thermo-Flux uses by default, and conditions it on the genotype. Now the potential has units, the sign of $\Delta_r G_j$ is a physical claim, and the learned part is a concentration that 191 measured yeast values can be compared against. Anchoring is what turns the thermodynamic term from a regularizer into a measurement, and the free-versus-anchored contrast is the experiment that says whether the tabulated energies carry information beyond the structural loop-freedom any potential gives.

Uncertainty is a latent, and this maps a solver step onto a gradient step. Thermo-Flux carries correlated uncertainty as $\Delta_r G^{\circ,\text{error}} = Qm$ with $m \sim \mathcal{N}(0,1)$ and Q the square-root covariance from eQuilibrator, and its Box 3 *infers* m by regression against measured concentrations and fluxes. That inference is what a latent variable does, so here m is a learned per-reaction offset under a Gaussian prior, fit by gradient descent instead of by a mixed-integer regression.

Q itself is not available: eQuilibrator is not installed and the shipped table is point values with no covariance, so the offset is isotropic rather than correlated. That is a real gap, recorded rather than approximated. The covariance would enter as one matrix multiply.

1.4 The exactness budget, and why it is a fifth arm rather than a footnote ✗

Exactness has to be spent on one constraint or the other, and which one matters is empirical:

Table 2. The exactness budget: which constraint each parameterization satisfies exactly. Exactness can be spent on the bounds or on mass balance, never both. The null-space head is also 2.7 times narrower, since it emits a latent of dimension 1,538 rather than one value per reaction.

parameterization	$Sv = 0$	box and directionality	head width
box, $v = \bar{v}^{\ell} + (\bar{v}^{u} - \bar{v}^{\ell})\sigma(z)$	soft, one weight	**exact**	4,131
null space, $v = \mathcal{N}z$ with $S\mathcal{N} = 0$	**exact**	soft, one weight	**1,538**

The null-space basis was computed by singular value decomposition and verified: $\mathcal{N}$ is 4131×1538 and $\max|S\mathcal{N}| = 5.3 \times 10^{-7}$, at float32 precision. The nullity of 1,538 matches the specification note exactly. The null-space head is also 2.7 times narrower, which is a real secondary benefit.

Merzbacher hit this from the other side and reported it: their deep models failed, which they attribute to "the fluxes being linearly correlated through $Sv = 0$." Those correlations are the constraint, which is the argument for putting the distribution on a latent rather than on per-reaction marginals.

1.5 The reconstruction: what the layer actually rests on ✗

Everything below is measured by `experiments/026-metabolism-flux/scripts/gem_audit.py` (`experiments.026-metabolism-flux.scripts.gem_audit`), which writes `results/gem_audit.json` and both figures. The audit exists because the failure mode of a constrained model is silent: a capacity constraint built on one organism-wide turnover number is a uniform rescaling of the flux box, not an enzyme constraint, and nothing in a loss curve distinguishes the two.

The network. yeast-GEM 9.0.2, loaded through cobrapy from the sha256-pinned checkout at `$DATA_ROOT/data/torchcell/yea`

Table 3. yeast-GEM 9.0.2 as constraint tensors, measured by `gem_audit.py`. The last row is the reason enzyme constraints are not optional: ten non-default bounds in 4,131 reactions means the stock model carries essentially no magnitude information, so turnover is the only quantity that can set a scale.

quantity	value
metabolites x reactions	2,806 x 4,131
nonzeros in S (density)	15,567 (0.134 %)
rank(S)	2,593
nullity	1,538
redundant balance rows	213
reversible / irreversible	1,668 / 2,463
exchange reactions	274
reactions with a gene-protein-reaction rule	2,709
catalytic units (AND-terms), of which multi-gene	3,728 / 275
genes	1,161
bounds that are not the $\pm$1000 default	**10**

That last row is the reason enzyme constraints are not optional. Ten non-default bounds in 4,131 reactions means the stock model carries essentially no magnitude information, so k_{cat} is the only thing that can set a scale.

Thermodynamics, and a provenance trap worth naming. The energies are in `data/databases/model_metDeltaG.csv` and `model_rxnDeltaG.csv`, which the model's own `code/missingFields/loadDeltaG.m` uses to populate its `metDeltaG` / `rxnDeltaG` fields. They are **absent from the SBML**, so a model loaded through `cobra.io.read_sbml_model` has no thermodynamics at all, which is why nothing in torchcell had read them before.

The file uses two missing-value conventions and only one of them is obvious. Most gaps are the sentinel `10000000`; a further 51 metabolites and 120 reactions are the literal string `NaN`. Filtering only the sentinel leaves the NaNs, which propagate through every sum: a reaction touching one gets a NaN standard energy, its hinge contributes NaN to the loss, and the run produces NaN gradients while the coverage number still reads a healthy 87 %. Rejecting both reproduces the model's own curation counts.

Table 4. The two missing-value conventions in the shipped thermodynamic tables, and what filtering only one of them costs. A sentinel-only filter leaves literal NaN values that propagate through every sum and produce NaN gradients, while the reported coverage still reads a healthy 87 percent. Rejecting both reproduces the model's own curation counts.

entity	sentinel-only filter	sentinel + NaN	the difference
metabolites	2,440 (87.0 %)	**2,389 (85.1 %)**	51 literal NaN
reactions	3,330 (80.6 %)	**3,210 (77.7 %)**	120 literal NaN

The two shipped tables do not agree with each other. Summing the metabolite table over S gives a standard reaction energy for the 3,505 reactions all of whose participants are known; comparing those against the shipped reaction column on the 3,204 reactions where both exist gives a median absolute residual of **9.53 kJ/mol**, 95th percentile **97.3**, maximum **396.1**. At 30 °C, $RT = 2.52$ kJ/mol, so the median disagreement is about $3.8\,RT$, a factor of roughly 45 in equilibrium constant. The two columns were produced by different routes and are not interchangeable.

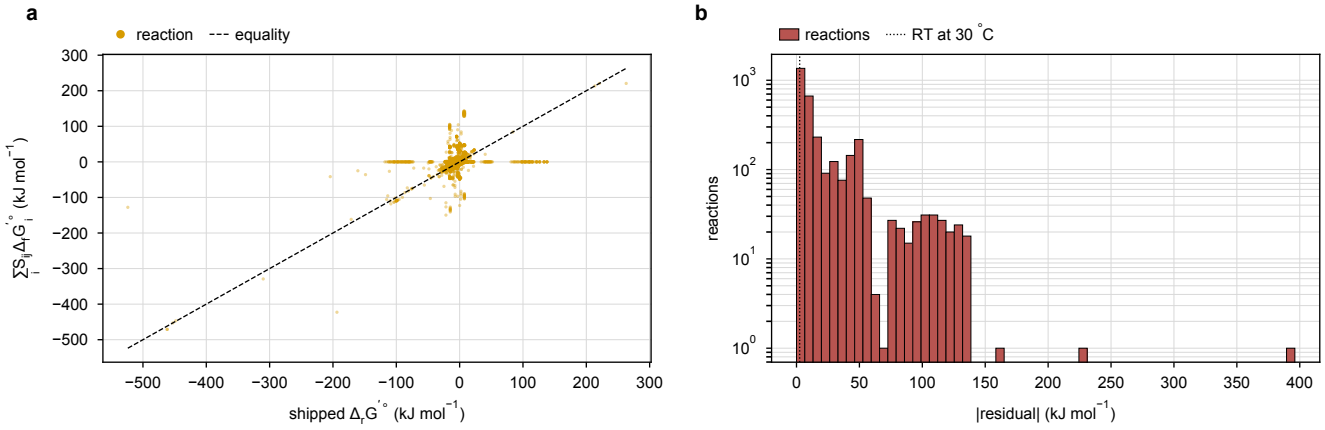


Figure 1. The two shipped thermodynamic tables do not agree, and the disagreement has structure. (a) The standard reaction energy recomputed by summing formation energies over the stoichiometry against the value shipped in the reaction table, on the 3,204 reactions where both exist; the dashed line is equality. The horizontal arm, where the recomputed value is exactly zero, is entirely transport reactions. (b) The absolute residual, with the dotted line at the thermal energy $RT = 2.52$ kJ/mol for 30 degrees Celsius. The median disagreement of 9.53 kJ/mol is about $3.8\,RT$, a factor of roughly 45 in equilibrium constant; the 95th percentile is 97.3 and the maximum 396.1. Drawn by `experiments/026-metabolism-flux/scripts/gem_audit.py`.

The disagreement is a cross, not a cloud, and one arm of it is the transport term. The scatter has a horizontal arm where the recomputed value is exactly zero while the shipped value ranges over ±150 kJ/mol, and a vertical arm with the opposite pattern. Testing the obvious hypothesis:

Table 5. The disagreement between the two shipped energy tables is a cross, and one arm of it is the transport term. Summing formation energies over a transport reaction cancels exactly, because the same species appears on both sides, so a transport reaction's whole standard energy is its transport term. Every reaction in the first row is multi-compartment and none in the second is.

pattern	n	multi-compartment
recomputed = 0, shipped > 5 kJ/mol	441	**441 of 441**
shipped = 0, recomputed > 5 kJ/mol	321	**0 of 321**

Summing formation energies over a transport reaction cancels exactly, because the same species sits on both sides: $\sum_i S_{ij} \Delta_f G'^\circ_i = 0$. So a transport reaction's entire standard energy **is** its transport term, and the shipped column carries the driving force the metabolite sum structurally cannot.

This both explains the gap and closes it. The transport term $\Delta_r G^t_j = -N_H RT \ln(10) \Delta\text{pH} - Fq\Delta\Phi$ cannot be computed from scratch without eQuilibrator species distributions, but it does not have to be: it can be **read off the model's own curation** for exactly the reactions whose recomputed energy is degenerate. `FluxLayerConfig.use_shipped_transport_delta_g` does this, populating 874 nonzero transport terms out of 1,099 qualifying reactions. It is off by default and off in every arm reported below, because it changes what the thermodynamic term asserts rather than tuning it, so it belongs in a named arm rather than in a silent default.

What it does not fix. The 2,021 single-compartment reactions still disagree by a median 10.2 kJ/mol, about $4\,RT$, and transport explains none of that. The anchored mode recomputes from the metabolite table and masks any reaction with an unknown participant, which is the more conservative of the two routes, and the residual disagreement is an open provenance question about the genome-scale model rather than about this layer.

Kinetics, and this is the headline gap. Molecular weight resolves completely; turnover does not. The resolution policy that produces these numbers, experimental before predicted before an organism default with a provenance tag on every value, is `torchcell.metabolism.parameters`.

Table 6. What each parameter class rests on, measured against yeast-GEM 9.0.2. Molecular weight resolves completely and turnover almost not at all. The 4.0 percent is the binding constraint on the entire enzyme layer, and it is measured in the best-curated eukaryote available.

parameter	source	coverage
molecular weight	SwissProt table shipped with the model	**1,161 / 1,161 (100 %)**, median 50.1 kDa
k_{cat}	Open Enzyme Database, *S. cerevisiae* slice	**148 / 3,728 catalytic units (4.0 %)**
measured concentration	YMDB table shipped with the model	191 / 2,806 metabolites (6.8 %)
$\Delta_f G'^{\circ}$	model's own table	2,389 / 2,806 (85.1 %)

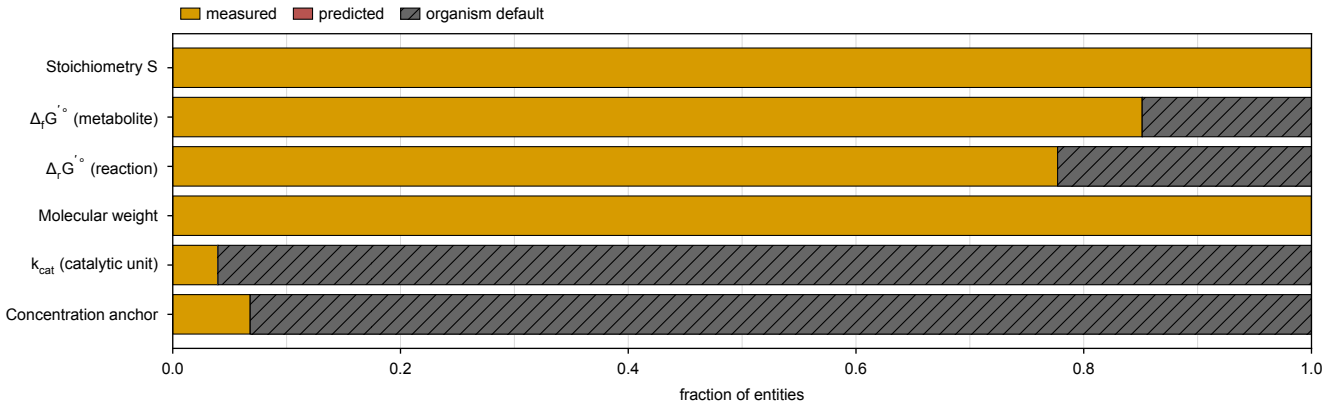


Figure 2. What each parameter class rests on: measurement, prediction, or an organism default. Stoichiometry and molecular weight are complete and formation energies cover most metabolites, but turnover is resolved for only 4.0 percent of catalytic units from the Open Enzyme Database and nothing is predicted yet, so the predicted band is empty in every row. The organism-default remainder carries a black hatch. A capacity constraint built on a default turnover is a uniform rescaling of the flux box rather than an enzyme constraint, and no loss curve distinguishes the two. Row by row: stoichiometry 1.000, $\Delta_f G'^{\circ}$ 0.851, $\Delta_r G'^{\circ}$ 0.776, molecular weight 1.000, k_{cat} 0.040, concentration anchor 0.068. Drawn by `experiments/026-metabolism-flux/scripts/gem_audit.py`.

Four percent, in the best-curated eukaryote there is. The entire Open Enzyme Database *S. cerevisiae* slice is 1,126 rows against 1,161 metabolic genes and 4,131 reactions. The remaining 96 % of catalytic units currently take an organism default of 10.3 s^{-1}, which is the median of the values that did resolve rather than a literature constant, and which is tagged `organism_default` per unit so a "published only" ablation is a filter rather than a re-run. This number is the argument for sequence-based prediction, and it is not a yeast problem, it is a floor: any other yeast or bacterial chassis has less.

A join bug found by hit rate, worth recording as a method. The SwissProt table's `gene_id` column is an **alias list**, not a standard-plus-systematic pair. `RAM2 YKL019W` is two tokens, but `ERG20 BOT3 FDS1 FPP1 YJL167W J0525` is six with the systematic name fifth, and token counts across the file run from 1 to 11. Taking the last token matched 430 of 1,161 genes; taking the first matched 38; selecting the token that matches a systematic open-reading-frame pattern matched 1,161 of 1,161. Nothing errored in any of the three cases. The check that caught it was joining against a known key set and looking at the hit rate, not inspecting the first few rows. It also moved measured k_{cat} coverage from 1.3 % to 4.0 %, so the 4 % above is the post-fix number.

A reproducibility check that passed. Re-fetching the Open Enzyme Database slice today produced a file byte-identical to the July 2026 capture, sha256 `0e4aca9e0872e5b19b7c325dc83c64cc43c9f2fba0d535d417e10a925e683398`, 1,126 records. Thirty-seven days, no drift.

The gaps, stated rather than approximated. The uncertainty covariance Q is unavailable, so the Thermo-Flux error latent is isotropic rather than correlated. The transport term was a gap too and is now recoverable, as the delta-G section above shows. Both are recorded in `FluxLayer.coverage_report()` so a run cannot report feasibility without also reporting what it did not model.

1.6 Closing the deferred metabolite-identifier gap ✗

All 19 Mülleder amino acids resolve to cytosolic yeast-GEM species, which is what the loader deferred as `target_metabolite_ids`: alanine `s_0955` through valine `s_1056`, tyrosine at `s_1051`. That mapping is what lets the metabolome head be mechanistic rather than another projection, since the model reads its own predicted turnover at exactly the species that were measured.

1.7 Porting to another organism, and why kinetics is the only real barrier ✗

The constraint layer is a pure function of a genome-scale model. There is no yeast constant in `torchcell/metabolism/constraint` (`torchcell.metabolism.constraints`) or `flux_layer.py` (`torchcell.metabolism.flux_layer`); the organism-specific surface is a model, a compartment table, and two scalars.

Table 7. The organism-specific surface of the constraint layer. Every row but the last is a lookup. Formation energies are chemistry and transfer unchanged; turnover and affinity do not, which is why the parameter layer is a resolution policy rather than a table.

ingredient	organism-specific?	where it comes from
S, bounds, gene-protein-reaction rules	yes	the new organism's model
$\Delta_f G'^{\circ}$	**no**, this is chemistry	a shared thermodynamic table
molecular weight	yes, but trivially	the proteome
compartment pH, ionic strength, $\Delta\Phi$	yes	Thermo-Flux Table 1 gives values for *E. coli*, *S. cerevisiae* and *Arabidopsis*
P_{avail}, g_{lim}	yes, two scalars	literature
k_{cat}, K_M	**yes, and this is the barrier**	see below

Everything except the last row is a lookup. The last row is 4 % covered in *S. cerevisiae* and worse everywhere else, which is why the parameter layer is written as a **policy** with a per-value provenance tag rather than as a table.

Resolution order, and database-first is a correctness requirement rather than a preference. A predictor trained on BRENDA will reproduce values it memorized, so using a prediction where a measurement exists both discards information and inflates any apparent agreement between the two.

1. **experimental**: BRENDA and the Open Enzyme Database, joined on UniProt accession and substrate, selected at the assay temperature nearest the phenotype's 30 °C;
2. **predicted**: a sequence-based model fills the gap;
3. **organism default**: reported as coverage, never presented as a measurement.

The three predictors, registered by capability rather than hardcoded.

Table 8. The three sequence-based kinetic predictors the parameter layer is built around. RealKcat is the one that closes the affinity gap, which matters because promiscuity is a K_M effect: Wu et al. (2026) measured underground reactions at roughly twice the K_M with indistinguishable k_{cat}, so a turnover-only model routes promiscuous flux at full native capacity for free.

predictor	inputs	emits
KcatNet	protein sequence + substrate SMILES	k_{cat}
RealKcat	protein sequence + substrate SMILES	k_{cat} **and** K_M
DEKP	sequence + optional structure file, graph network over pretrained language models	k_{cat}

RealKcat is the one that closes the affinity gap, and affinity is what makes promiscuity cost something: Wu et al. (2026) measured underground reactions at about 2x higher K_M with **indistinguishable** k_{cat}, so a k_{cat}-only model routes promiscuous flux at full native capacity for free. DEKP is the right choice when a predicted structure exists and the sequence is far from anything in the training set, which is the usual situation for a novel chassis.

`KcatPredictor` is a protocol with a registry, so a run records **which predictor produced which value**. Wiring an actual checkpoint in is deliberately a separate, GPU-bearing step and is not done here. Note that GECKO 3.0 reports doing the same thing, "incorporates deep learning-predicted enzyme kinetics" to cover organisms lacking experimental data, so this is the field's converged answer rather than a novel one.

Carrying uncertainty rather than a point value is the intended next step and is already shaped for it: a predicted k_{cat} that is too low forbids feasible flux and one that is too high is vacuous, so the predictor's posterior quantile q belongs in the box. $q = 0.1$ means the model may use only capacity it is confident exists, $q = 0.5$ is the usual point estimate, and resampling q per forward pass makes the box itself stochastic, which composes with the amortized-sampler framing.

1.8 Always emit fluxes, or sample on demand? One switch, and it is not architectural ✗

The question of whether the forward pass should always output fluxes or be sampled when needed turns out not to be a fork in the architecture. Both are the same layer with `FluxLayerConfig.stochastic` flipped, because the box is applied identically either way.

Table 9. Deterministic versus stochastic flux head. Both are the same layer with one configuration flag flipped, because the box applies identically either way. The flux is always produced, since the phenotype heads read it; what the flag changes is whether repeated calls on one genotype vary.

	deterministic head	stochastic head
z_j	emitted by the reaction network	drawn from $q_\phi(z \mid H_{\text{pert}})$
v is	one point **selected** by the objective's implicit preference	a random variable supported on the feasible set
cost	one forward pass	S forward passes for S draws
what it replaces	parsimonious flux balance analysis	Markov-chain flux sampling

The flux is always produced, since the phenotype heads read it. What `stochastic` changes is whether repeated calls on one genotype vary. So the practical policy is to train deterministic, which is cheaper and better conditioned, and to sample only where a distribution is the deliverable: reporting an interval, propagating kinetic-parameter uncertainty, or scoring a design whose label does not exist yet.

The trap, and it is the reason the latent is shared. A per-reaction marginal distribution is not a distribution over flux vectors. $Sv = 0$ constrains the joint, so drawing each v_j from an independent marginal almost surely violates it. Sampling one latent and mapping it to all 4,131 coordinates at once is what keeps them coupled, and it is the same point Merzbacher reached from the other direction when their deep models failed "attributed to the fluxes being linearly correlated through $Sv = 0$."

Flux variability analysis is the reference, and it is label-free. Running it on the wild type at 90 % of optimum, by `scripts/fva_reference.py` (`experiments.026-metabolism-flux.scripts.fva_reference`):

Table 10. Wild-type flux variability analysis at 90 percent of optimum, the reference an amortized sampler is scored against. The licensed set excludes reactions the constraints leave free over the full range, where narrowing would mean nothing. The count of 2,818 reproduces the specification note's independently derived figure exactly.

quantity	value
wild-type growth	0.0858 h^{-1}
blocked reactions, width 0	1,286
width $\leq$ 1, the licensed set	**2,818**
width $\leq$ 10	3,671
width $\geq$ 1000, i.e. unconstrained	452
median width	0.118

The 2,818 figure reproduces the specification note's independently derived count exactly. Restricting the comparison to that set matters: a reaction FVA leaves free over the full ± 1000 range is not one where narrowing means anything, it is one the constraints say nothing about. The evaluation is then $\text{width}_{\text{FVA}}(j) - \text{width}_{\text{posterior}}(j)$ per licensed reaction, which needs no phenotype label and can therefore be run on double deletions where no production measurement exists.

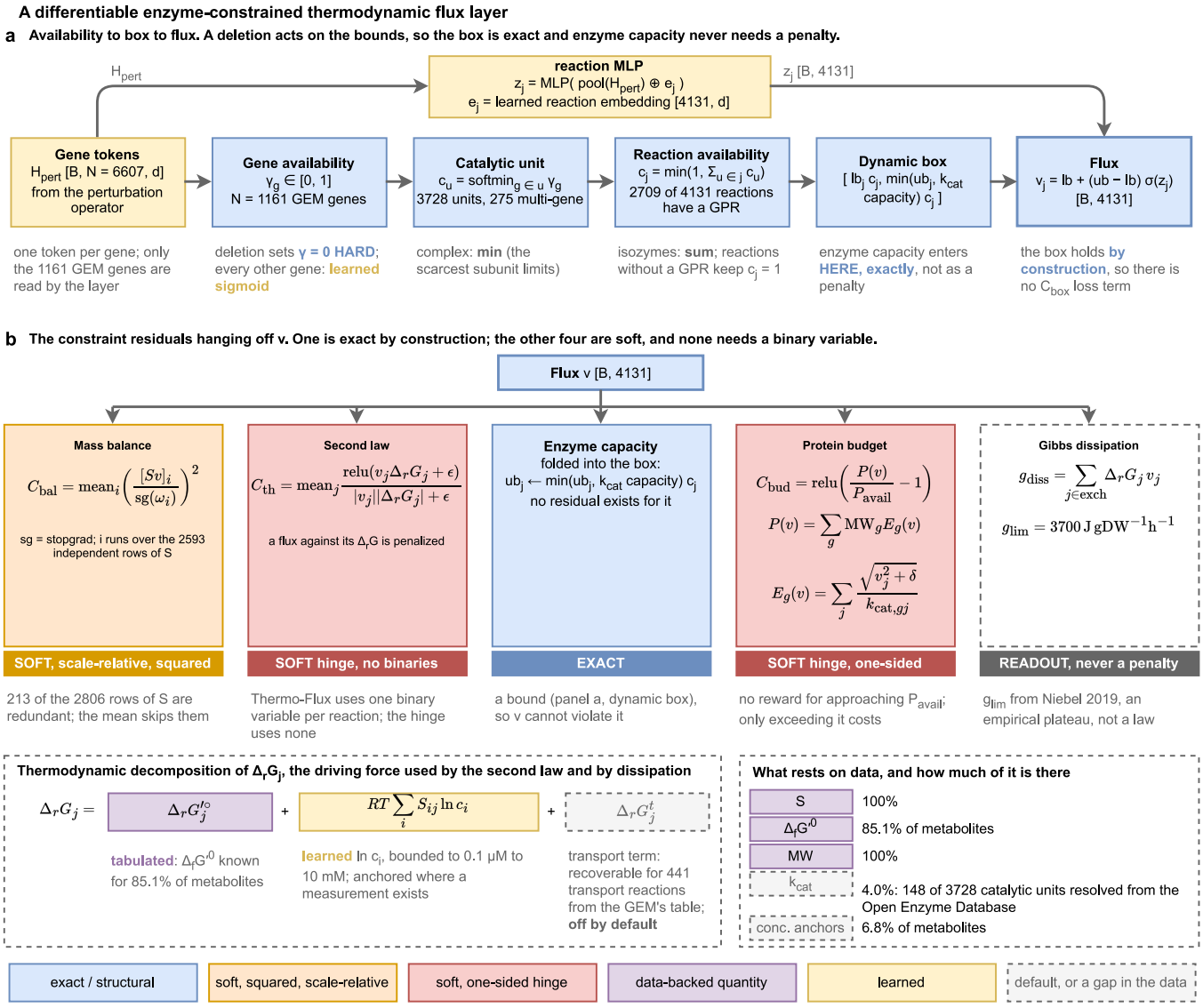


Figure 3. The flux layer, from gene tokens to a feasible flux vector and its residuals. (a) A deletion sets gene availability to zero as a hard fact, not a learned gate; complexes take a softmin over subunits and isozymes sum, and enzyme capacity enters the bounds rather than the loss, so the box holds by construction and contributes no loss term. (b) Five residuals hang off the flux. One is exact, three are smooth penalties, none requiring a binary variable, and Gibbs dissipation is a readout rather than a penalty (see the 2026.09.08 correction below). The insets give the three-term decomposition of the reaction driving force and the fraction of each input that is measured rather than defaulted.

The embedded file is `metabolism-flux-layer.vector.svg`, not draw.io's own SVG export. draw.io writes its labels as HTML `foreignObject` elements, which `rsvg-convert` cannot draw, so the note-to-PDF path renders that export with every label truncated and a "Text is not SVG, cannot display" overlay, and exits 0. Round-tripping draw.io's PDF export through `pdftocairo -svg` outlines the text into paths, which renders identically everywhere and stays zoomable. `notes/assets/publish/scripts/drawio_vector_svg.sh` does this and fails loudly if any `foreignObject` survives.

1.9 The diagnostic experiment ✗

Two phenotypes, both single-deletion panels, both already built into the `fig6_pigment_transfer` dataset at 4,930 aggregated genotypes.

Table 11. The two phenotypes, both single-deletion panels already built into the `fig6_pigment_transfer` dataset. Betaxanthin has a replicate-based ceiling and the amino-acid panel has none, since Mülleder reports one replicate per strain with no standard error.

phenotype	source	n	shape	ceiling
betaxanthin	Cachera 2023, CRI-SPA corrected fluorescence	4,735	scalar, population-centered	$r = 0.914$
amino acids	Mülleder 2016, intracellular mM	4,678	vector(19)	none, $n_{\text{rep}} = 1$

The heads are mechanistic in the flux arms and pooled in the baseline, which is what makes the comparison mean anything. They are built on `CellGraphTransformerMetabolism` (`torchcell.models.cell_graph_transformer_metabolism`), which holds the flux layer and dispatches each head by kind.

- `mulleder19` reads the model's own turnover at the 19 measured species: $\hat{y}_k = a_k \log(1 + \omega_{i_k}(v)) + b_k$. Turnover is not concentration, and the per-metabolite affine is where that gap is admitted: a steady-state model has no residence time in it, so the claim is monotone, not calibrated, and the metric to read is a correlation rather than an error in mM. The affine is per metabolite because the 19 pools differ by orders of magnitude and one shared slope would let the largest set the scale.
- `betaxanthin` is a product the network does not contain, since its route is a heterologous cassette and no column of S carries it. What the network does carry is the precursor supply, so the head reads $\hat{y} = w^\top v + u^\top \log(1+\omega_{\mathcal{P}}(v)) + b$ with $\mathcal{P}$ the named aromatic precursors including cytosolic tyrosine. The dense $w^\top v$ term is kept alongside so the precursor claim can be ablated against an unrestricted read of the same flux vector rather than assumed.

The metabolome head has almost no capacity, and that is deliberate. It holds 38 parameters, two per measured amino acid, against the pooled baseline's multilayer perceptron. Any correlation it achieves therefore comes from the flux vector rather than from the head, which is what makes it a test of the flux layer instead of a test of a readout. The cost is that a null result is ambiguous between "the flux carries no signal" and "an affine in $\log \omega$ is the wrong link function", and only the first of those is about metabolism. Reading the pooled arm alongside is what separates them.

Five arms, each changing one thing relative to the one above, so a difference is attributable. Three seeds each, identical splits within a seed.

Table 12. The five arms, each changing exactly one thing relative to the arm above it. `pooled` to `flux_off` asks whether routing a prediction through a stoichiometric network helps at all; `flux_free` to `flux_anchored` asks whether the measured energies add anything beyond the loop-freedom any potential gives; `flux_anchored` to `flux_nullspace` is the exactness budget.

arm	what it adds
`pooled`	baseline: the existing multilayer perceptron over pooled gene tokens
`flux_off`	flux layer, availability chain, mass balance. No thermodynamics, no k_{cat}
`flux_free`	+ learned potential, so loop-freedom without tabulated energies
`flux_anchored`	+ tabulated $\Delta_f G'^{\circ}$, enzyme capacity, budget, dissipation
`flux_nullspace`	the same, with the exactness budget spent on $Sv = 0$ instead of the box

`pooled` to `flux_off` asks whether routing the prediction through a stoichiometric network helps at all. `flux_free` to `flux_anchored` asks whether the measured energies carry information beyond the structural loop-freedom any potential gives. `flux_anchored` to `flux_nullspace` is the exactness budget.

The repo's earlier stoichiometric model, `StoichHypergraphConv` (`torchcell.nn.stoichiometric_hypergraph_conv`), spends S the other way: the signed coefficients weight messages passed over the metabolic hypergraph, so the network shapes the representation and constrains nothing. Here S is a constraint on an emitted flux and never a message weight. That layer is not one of the five arms, so the two designs have not been compared.

Loss weighting had to be set from a measurement, not chosen. At initialization on a real batch the unweighted constraint sum is about 255 against a data loss of about 2, so at weight 1 the model spends every parameter on feasibility and none on the phenotype. Two terms were also reformulated rather than merely down-weighted, because their raw forms are not dimensionless: the second-law hinge is now divided by the available driving force, and the dissipation excess is a ratio rather than a square. The squared form evaluated to about 4×10^4 at initialization and drove the first run to NaN within one step.

1.10 The sharpest result so far: a sigmoid box cannot reach a sparse flux vector ✗

In every box-parameterized arm the mass-balance diagnostic $\text{median}_i |[Sv]_i|/\omega_i$ sits at **1.99** and does not move over training, while the second-law violation fraction falls steadily from 0.373 to 0.257 over the same epochs. The thermodynamic penalty is working and the balance penalty is not, and the reason is structural rather than a matter of weight.

2.0 is the maximum that statistic can take. With $\omega_i = \frac{1}{2}\sum_j |S_{ij} v_j|$, a metabolite that is only produced and never consumed gives $|[Sv]_i| = 2\omega_i$ exactly. A median of 1.99 says the median metabolite is completely unbalanced.

The mechanism is the parameterization. For the 2,463 irreversible reactions $v_j^\ell = 0$, so $v_j = 0$ requires $\sigma(z_j) = 0$, that is $z_j \to -\infty$. **Zero flux is an asymptote of the parameterization, not a point in it.** A real parsimonious flux solution is overwhelmingly sparse, so balanced solutions live exactly where a sigmoid box cannot go, and the optimizer's only route there is to drive thousands of logits to large negative values against weight decay.

Measured by `scripts/box_zero_reachability.py` (`experiments.026-metabolism-flux.scripts.box_zero_reachability`) on the real network at initialization:

Table 13. Flux magnitude and mass balance under both parameterizations, at initialization on the real network. Zero flux is an asymptote of a sigmoid box, so balanced solutions live where that parameterization cannot go. The null-space form is 70 times more likely to produce a near-zero flux without being asked.

quantity	box	null space
median $\lvert[Sv]_i\rvert/\omega_i$	1.27	**2.8e-08**
metabolites at the maximum imbalance of 2	29.4 %	0 %
metabolites below ratio 0.1	8.0 %	100 %
fluxes below 10^{-4} mmol gDW^{-1} h^{-1}	**0.17 %**	11.9 %
median $\lvert v_j\rvert$, mmol gDW^{-1} h^{-1}	11.0	9.0
box violation fraction	**0 %**	37.0 %

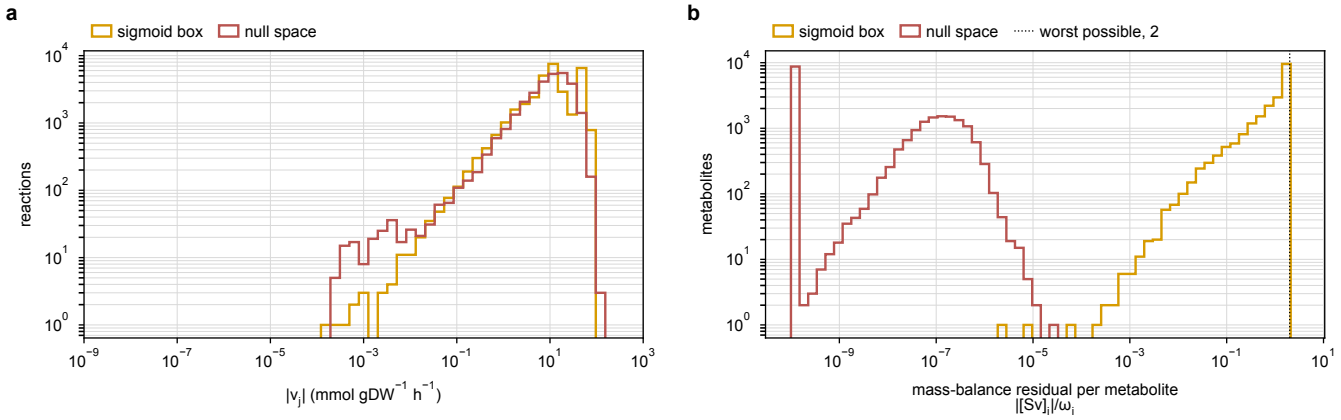


Figure 4. Why a sigmoid box cannot produce a sparse, mass-balanced flux vector. (a) The distribution of flux magnitudes at initialization; the null-space form reaches near-zero fluxes that the box does not, because zero flux is an asymptote of the sigmoid rather than a point in its range. (b) The mass-balance residual as a fraction of metabolite turnover. The box arm piles up against the dotted line at 2.0, which is the maximum the statistic can take and means the median metabolite is completely unbalanced, while the null-space arm sits eight orders of magnitude lower at float32 round-off: median 1.27 against 2.8e-08, with 29.4 percent of box metabolites exactly at the maximum against 0 percent for the null space. Both axes are logarithmic. Drawn by `experiments/026-metabolism-flux/scripts/box_zero_reachability.py`.

The right panel is the whole argument in one picture. The box arm's balance residuals pile up against the dotted line at 2.0, the maximum the statistic can take, while the null-space arm sits eight orders of magnitude lower at float32 round-off.

To put a reaction at 10^{-6} of its upper bound the logit must reach -13.8; at 10^{-9}, -20.7. Doing that for the roughly 88 % of reactions that should carry no flux means driving about 3,600 logits to -14 or beyond while weight decay pulls them back. The null-space parameterization is not merely exact on mass balance, it is also **70 times more likely to produce a near-zero flux** without being asked, because sparsity is not fighting the parameterization there.

The same cause shows up in the dissipation term. At the end of the anchored run the Gibbs dissipation rate is 5.54×10^5 J gDW^{-1} h^{-1} against Niebel's limit of 3.7×10^3, a 150-fold excess, because g_{diss} sums $\Delta_r G_j v_j$ over exchange reactions and the box keeps every one of them running near half capacity. Two of the four soft constraints are therefore failing for one structural reason rather than for two independent ones, which is worth knowing before either weight is touched.

What is working. The second-law hinge falls from 0.373 to 0.266 over 20 epochs, and the protein budget is satisfied with room to spare at 0.26 % of P_{avail}. Neither of those depends on driving fluxes to zero, which is consistent with the diagnosis.

The design consequence. If the box is kept, it needs an explicit zero rather than a heavier penalty: a gate, a hard-concrete mask, or a shifted sigmoid with a flat region at the lower bound. That is a specific, testable next change, and it is the thing this diagnostic bought that a loss curve alone would not have.

1.11 Media: the FBA layer works, the ontology objects do not reach it yet ✗

The question of whether the `Media` schema maps onto exchange bounds now has a measured answer, from `torchcell/metabolism/media.py` (`torchcell.metabolism.media`) and `scripts/media_schema_audit.py` (`experiments.026-metabolism-flux.scripts.media_schema_audit`). The mapping resolves each media com-

ponent to an exchange reaction through the model's own annotations, and reports coverage rather than silently dropping anything.

The four recipes work. All 271 exchange reactions are indexed, every component resolves, and every medium supports growth.

Table 14. The four media recipes mapped onto exchange bounds, with growth under each. Every component of every medium resolves to an exchange reaction, and each medium supports growth. Pairwise differences confirm the four collapse to one real axis, the 20 amino acids, plus one or two nucleobases.

medium	components	resolved	unresolved	growth, h^{-1}
SM	25	25	0	0.314
SC	47	47	0	0.543
SC-URA	46	46	0	0.539
YPD-approx	45	45	0	0.535

The pairwise differences confirm the prediction that our four media collapse to one real axis: SM against SC differs by 22 exchanges, the 20 amino acids plus adenine and uracil, while SC against SC-URA differs by exactly one, uracil. No bound ever differs in magnitude, only in presence.

The sourcing error is confirmed and costs a factor of three. Suthers et al. set supplements to a fixed 0.165 mmol gDW^{-1} h^{-1}, which is 5 % of their *default* 3.3 glucose uptake, and their own runs at glucose 10.0 do not rescale. Our older code computes `glucose_rate * 0.05`, giving 0.5 at glucose 10.0, or **3.03x the sourced value**. The growth consequence on SC is 1.194 h^{-1} sourced against 1.652 rescaled. The new module follows the source.

The ontology objects do not reach any of this, and that is the finding. All four datasets emit a name-only `Media` with **zero components**, so each maps to zero bounds and an infeasible model with every exchange closed:

Table 15. What the four datasets actually emit for their growth medium. Each is a name-only object with zero components, so it maps to zero bounds and an infeasible model with every exchange closed. The join from a dataset to a medium is currently a name string, not a component list.

loader	line	emitted
`cachera2023.py`	321	`Media(name="SC", state="solid", is_synthetic=True)`
`ozaydin2013.py`	361	`Media(name="SC-URA", state="solid", is_synthetic=True)`
`mulleder2016.py`	241	`Media(name="SM", state="solid", is_synthetic=True)`
`ohya2005.py`	317	`Media(name="YPD", state="liquid", is_synthetic=False)`

So the join from a dataset to a medium is currently **a name string**, not a component list. Pushing all eight hand-written media from the library through the same mapping gives growth 0.0 for seven and infeasible for the eighth, and `SM` has no ontology object at all.

What the schema is missing, precisely. A bound is a rate and a `Concentration` is an amount per volume, so the conversion needs fields that do not exist:

1. no molar mass or formula on `Compound`, so `2.0 percent_w/v` glucose cannot become mM;
2. no culture physiology (biomass density, volume, batch versus chemostat, dilution rate), so the uptake magnitude has to come from a model convention rather than from the record;
3. no aeration or atmosphere field, and oxygen is the single largest determinant of yeast flux;
4. water is never a recipe line, so the solvent must be injected by the adapter;
5. `composition_deferred` hides the entire mineral and vitamin base, so one unresolvable "yeast nitrogen base" line stands in for about 14 salts and 9 vitamins;
6. `Compound.chebi_id` is `None` on every component in the library, so the annotation join the design note names as the key currently carries 0 of 163 resolutions. The channel itself works: 47 of 47 SC components round-trip from the model's own ChEBI ids alone;
7. no transportability flag, so agar, canavanine, G418 and peptone are excluded by role inference rather than by a declared property.

Three of the four loaders also record `state="solid"` for plate screens while the recipes are liquid, and Ohya correctly records `is_synthetic=False` against the recipe's `True`. That last disagreement is the honest signal that YPD-approx is a stand-in rather than YPD, which is why it is named that way.

1.12 Literature captured for this work ✗

Table 16. Literature captured for this work. Nothing was written to Zotero. The closed-access GECKO 3.0 protocol is recorded as a provenance gap with every attempted retrieval route and its exact failure, plus a manual recipe.

key	status
`smithThermofluxGenerationAnalysis2026a`	already mirrored, read in full; the source for the second-law relaxation, the exemption set, the three-term decomposition, and the dissipation limit
`fangReconstructingOrganismsSilico2020`	**newly mirrored**, full text via an open-access author manuscript, MinerU OCR, sha256-pinned manifest
`chenReconstructionSimulationAnalysis2024`	**documented provenance gap.** GECKO 3.0 is closed access; ten retrieval routes tried and recorded with their exact failures, plus a manual recipe. Abstract only

Nothing was written to Zotero. The GECKO 3.0 abstract does confirm the direction taken here: it "incorporates deep learning-predicted enzyme kinetics" to cover organisms lacking experimental k_{cat}, so the predictor tier is the field's converged answer.

Fang et al. is worth one line beyond the mirror: their review names enzyme complex stoichiometry and k_{cat} as **the two bottlenecks** in metabolic and expression model reconstruction, and calls systems-level k_{cat} elucidation an open area. The 4.0 % coverage measured here is that bottleneck with a number on it.

1.13 Reproducing everything ✗

Every command runs from the worktree root with `PYTHONPATH=$PWD` and the torchcell environment interpreter. Scripts live in `experiments/026-metabolism-flux/scripts/` and outputs land in the sibling `results/`, so only the bare names vary:

```
PYTHONPATH=$PWD ~/miniconda3/envs/torchcell/bin/python \
    experiments/026-metabolism-flux/scripts/<script>
```

Table 20. Every artifact in this note and the committed script that regenerates it. All commands run from the worktree root with `PYTHONPATH=$PWD` and the torchcell environment interpreter; script paths are relative to `experiments/026-metabolism-flux/`.

step	script	writes
parameter audit	`gem_audit.py`	`gem_audit.json`, Figure 2, Figure 1
media audit	`media_schema_audit.py`	`media_schema_audit.json`
FVA baseline	`fva_reference.py`	`fva_wildtype.csv`, `fva_census.json`
box reachability	`box_zero_reachability.py`	`box_zero_reachability.json`, Figure 4
the arm sweep	`train_flux.py --arms ... --seeds 42,7,1234`	`flux_arms_gpu{0,1}.json`
figures + summary	`plot_flux_arms.py`	`flux_arms_summary.json`, Figure 5
flux sampling	`flux_sampling_demo.py`	`flux_sampling_demo.json`
diagram vector	`drawio_vector_svg.sh` (in `notes/assets/publish/scripts/`)	Figure 3

Tests: `pytest tests/torchcell/metabolism/test_flux_layer.py` (13 property tests, one requiring the yeast-GEM checkout). They assert the claims rather than the outputs: the box holds exactly, a deletion zeros its reaction, an unannotated reaction is never deleted, the null space balances mass to machine precision, no residual is NaN or out of scale, and the gradient reaches the gene tokens.

1.14 Results: five arms, three seeds, 20 epochs ✗

The sweep runs from `scripts/train_flux.py` (`experiments.026-metabolism-flux.scripts.train_flux`) and every number and figure below is reduced from its output by `scripts/plot_flux_arms.py` (`experiments.026-metabolism-flux.scripts.plot_flux_arms`).

Peak validation Pearson, identical splits within a seed. Betaxanthin's replicate-based noise ceiling is 0.914, so every number here is between 1 % and 12 % of what the measurement supports.

Table 17. Peak validation Pearson per arm, three seeds, identical splits within a seed. Betaxanthin's noise ceiling is 0.914, so every value is between 1 and 12 percent of what the measurement supports. No between-arm mean difference clears the seed spread; the only comparison that separates is the variance, and that is post-hoc.

arm	betaxanthin per seed	mean	sd	amino acids, mean
`pooled`	0.041, 0.077, 0.059	0.059	0.018	+0.001
`flux_off`	-0.001, 0.093, -0.071	**0.007**	**0.083**	-0.006
`flux_free`	0.102, 0.095, 0.053	0.083	0.027	+0.008
`flux_anchored`	0.079, 0.090, 0.091	0.087	**0.007**	+0.002
`flux_nullspace`	0.140, 0.100, 0.083	**0.108**	0.029	+0.004

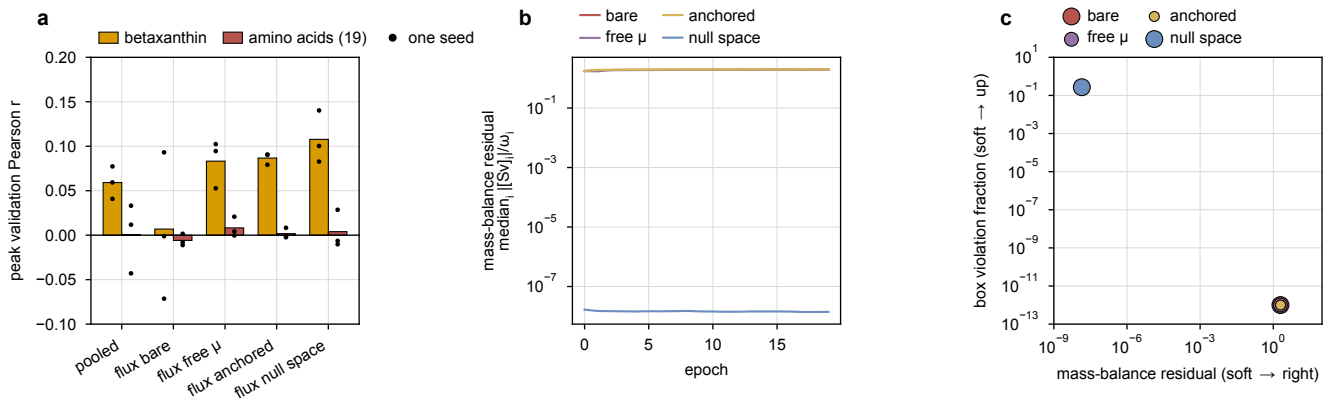


Figure 5. Five arms, three seeds, 20 epochs. (a) Peak validation Pearson, with each seed drawn as a black point rather than an error bar, because three seeds do not describe a distribution. No between-arm mean difference on betaxanthin clears the seed spread, and the amino-acid panel is null in every arm including the pooled baseline. The replicate-based betaxanthin ceiling is 0.914, so every bar here is under an eighth of what the measurement supports. (b) The mass-balance residual over training on a logarithmic axis, where the three box arms overlie each other near the maximum of 2 and the null-space arm sits at round-off. The pooled baseline has no flux layer and so appears in a only. (c) The exactness budget. `flux bare`, `flux free` and `flux anchored` land on one point by construction and are drawn as nested rings there, largest first; the null-space arm sits at the opposite corner, and neither corner reaches the origin. Drawn by `experiments/026-metabolism-flux/scripts/plot_flux_arms.py`.

On betaxanthin, no arm-to-arm difference clears the noise. The largest gap is `flux_nullspace` over `pooled` at +0.049 against a difference standard error of 0.020, so 2.4 standard errors at three seeds. `flux_anchored` over `pooled` is +0.028 at 2.5. Both are suggestive and neither is a result: the 019 replicate study established that gaps below about 0.04 are noise on this substrate, and one of these two is below that line while the other sits on it. **They are reported as trends and nothing is claimed from them.**

The one comparison that does separate is a variance, not a mean. `flux_anchored` has a seed spread of 0.0065 against `flux_off`'s 0.0826, a variance ratio of 161. Under an F test at two and two degrees of freedom that clears the 1 % level, and unlike the mean differences it is a large effect measured on a quantity three seeds can actually estimate. The honest caveat is that it is **post-hoc**, being the largest of ten pairwise variance ratios, so a Bonferroni-corrected threshold would want about 199 rather than 161. Read it as strong-but-not-confirmed.

The ordering it suggests is worth stating even though the means do not separate. A bare flux layer with no thermodynamics and no enzyme constraints is the worst and by far the least stable arm, below the pooled baseline it replaces. Adding a potential recovers it, and adding the tabulated energies plus enzyme capacity makes it reproducible. **Routing a prediction through a stoichiometric network is not what helps; constraining that network is.** That is a testable claim and the next round should test it, because it is exactly the claim these three seeds cannot settle.

The amino-acid panel is a clean null across every arm, including the pooled baseline. Arm means run from -0.006 to +0.008 while individual seeds swing between -0.043 and +0.033, so the between-arm differences are an order of magnitude smaller than the within-arm spread. Two readings remain open and this experiment does not separate them: the flux vector may carry no information about amino-acid pools, or an affine in $\log \omega$ may be the wrong link function for a 38-parameter head. Only the first is a statement about metabolism. What can be said is that the pooled multilayer perceptron does no better, so the mechanistic head is not losing to a readout with more capacity.

Feasibility behaved exactly as the parameterization analysis predicted, which is the part of this experiment that did work as designed:

Table 18. Feasibility at the end of training, which behaved exactly as the parameterization analysis predicted. The three box arms sit at one corner and the null-space arm at the other, and neither reaches the origin. The null-space arm's second-law violation near 0.5 shows that spending exactness on mass balance costs the thermodynamic term as well as the bounds.

arm	mass balance	box violation	second-law violation
`flux_off`	1.97	0	term disabled
`flux_free`	1.96	0	0.199
`flux_anchored`	1.98	0	0.266
`flux_nullspace`	**1.4e-08**	**0.27**	0.45

The three box arms sit at one corner and the null-space arm at the other, and neither reaches the origin. The null-space arm's second-law violation of 0.45 is near the 0.5 of a coin flip, so spending exactness on mass balance costs the thermodynamic term as well as the bounds.

A metric defect found and fixed mid-sweep. The null-space arm returned `nan` for betaxanthin in 8 of 20 epochs at

one seed and exactly `0.0` in 3 more. Both are the same event: the head collapsed onto a near-constant prediction, and an absolute variance floor of 1e-12 sits in the middle of the residual spread such a collapse leaves, so the same failure was reported two different ways and neither said what happened. The guard is now relative to the target's spread, non-finite pairs are excluded and counted, and the whole arm was re-run under the corrected metric. **The numbers in the table above are the corrected ones.** The lesson generalizes past this experiment: a collapse detector with an absolute threshold does not detect collapses.

1.15 Amortized flux sampling, and what the width comparison does and does not show ✗

`scripts/flux_sampling_demo.py` (`experiments.026-metabolism-flux.scripts.flux_sampling_demo`) trains the anchored arm with a stochastic head for 12 epochs, then draws 128 flux vectors for each of 32 genotypes. Every draw is one forward pass and every draw lands inside its genotype's box, so no rejection step is involved.

Table 19. Amortized flux sampling against the flux-variability reference. The narrowing is not evidence that data added information: a collapsed sampler emits zero-width intervals and would score 100 percent here. The missing half of the evaluation is coverage, the fraction of held-out observations falling inside the predicted interval.

quantity	value
FVA-licensed reactions, width in $(0, 1]$	1,532
reactions where the model interval is narrower	868 (56.7 %)
median model interval width	0.0081
median FVA interval width	0.1114

The 1,532 is the 2,818 reactions of width ≤ 1 minus the 1,286 blocked ones, which is a consistency check on the whole pipeline rather than a new number.

The narrowing is not evidence that the data added information, and it must not be read that way. A sampler that has collapsed produces a zero-width interval and would score 100 % on this comparison. With 12 epochs and a betaxanthin correlation of 0.036 this model is barely trained, so a median width 14 times narrower than FVA's is at least as consistent with an under-dispersed posterior as with a genuine reduction in uncertainty. **The missing half of the evaluation is calibration**: the fraction of held-out observations falling inside the predicted interval. Width without coverage is not information, and running the coverage check is the first thing to do before this number is quoted anywhere.

What the demonstration does establish is the mechanism and its cost. A flux distribution per genotype is 128 forward passes rather than a Markov chain run to mixing, a double deletion is the same forward pass as a single, and the comparison needs no phenotype label, so it can be run on genotypes whose production has never been measured.

1.16 Next changes, in the order they should happen ✗

1. **Give the box an explicit zero.** The reachability measurement says a sigmoid box cannot produce a sparse flux vector, and that is the single largest structural defect. A gate, a hard-concrete mask, or a shifted sigmoid with a flat region at the lower bound are the three candidates; the diagnostic to score them against already exists. 1b. **Add the calibration half of the sampler evaluation.** Interval width alone is satisfied by a collapsed posterior, so the coverage fraction has to be reported beside it before any narrowing is described as information gained. 1c. **Re-run the arms at more seeds and more epochs.** Every mean difference here is inside the noise, and the specific claim worth testing is the one the variances suggest: that constraining a flux layer, not merely having one, is what makes it work.
2. **Run the transport-term arm.** `use_shipped_transport_delta_g` is implemented and tested but was off in every arm reported here, so whether recovering 874 transport driving forces changes the thermodynamic term's behavior is currently unmeasured.
3. **Wire one k_{cat} predictor.** 4.0 % coverage is the binding constraint on the enzyme layer, and RealKcat is the one that also closes K_M. The provenance tag makes the published-only ablation free once the values exist.
4. **Give the loaders component-bearing `Media` objects.** The mapping works and the recipes grow; the join is a name string. Populating `Compound.chebi_id` alone would switch the annotation channel from 0 to 47 of 47 resolutions on SC.
5. **Compare against Flux Cone Learning as a distribution over flux vectors, not as a classification score.** The earlier plan was to reproduce their 3-class accuracy on their 811 metabolic genes, and that is the wrong comparison: it scores a downstream classifier rather than the flux, and their own reported margin over the majority class is 2.6 points. The meaningful comparison uses the samples they already released. Merzbacher's cones are mirrored at `databases/fcl/paper_merzbacher_2025/` from Zenodo `10.5281/zenodo.15761895`: 1,159 single deletions, 124 samples per cone, 143,716 samples by 4,130 fluxes. So fit one of our models, draw from the stochastic head on the same deletions, and ask whether the two flux-vector distributions agree at all, per reaction and per genotype. That is a direct comparison against an existing dataset and it needs no phenotype label. **State the caveat every time:** their chains target a uniform distribution over the feasible polytope and ours targets whatever the data and priors induce, so agreement is informative and disagreement is not automatically a failure.

1.17 What is not claimed ✗

The runs are short, small, and diagnostic: 20 epochs, hidden dimension 32, two transformer layers, learnable gene embeddings, three seeds. **They do not establish that a flux layer improves production prediction.** Every between-arm mean difference on betaxanthin is inside the seed spread, the amino-acid panel is null everywhere, and the sampler's interval narrowing has no calibration check behind it yet.

For scale, the previous round reached 0.128 on betaxanthin with 80 epochs and ESM2 features; these arms reach 0.108 at a quarter of the epochs with learnable embeddings, which says the setups are comparable and says nothing about the flux layer. The ceiling is 0.914.

What the work does establish is structural and was measured rather than argued: what the constraint layer rests on, where the parameterizations trade exactness, why a sigmoid box cannot balance mass, where the two shipped thermodynamic tables disagree and why, and that the media ontology does not yet reach the flux layer.

2 2026.09.03 - The arm comparison against a calibrated null, and the kinetics inputs ✗

Four GilaHyper GPUs, jobs 1576 to 1579, ran 98 of 98 cells in 10.5 to 11.0 h each. Every grid completed. Scored by `experiments.026-metabolism-flux.scripts.analyze_sweep`, generated by `experiments.026-metabolism-flux.scripts.sweep_flux`.

2.1 Why this was run instead of a longer budget ✗

The 20-epoch comparison in the section above reported each arm by its maximum validation Pearson over epochs. Three measurements say that comparison could not distinguish anything:

- across all 18 banked runs, validation Pearson peaks at a **median epoch of 4.5** out of 20, and the mean of the final five epochs is at or below zero in 13 of them;
- the reported score is a **maximum over epochs**, an upward-biased order statistic whose bias grows with the number of epochs run;
- the validation split carries a median of **339 betaxanthin observations**, so a single Pearson has null width $1/\sqrt{n-3} = 0.0546$, while the arms were separated by 0.004 to 0.08.

A longer budget makes an overfitting model worse, and it cannot fix a statistic whose spread exceeds the effect. So the window went to replication and to calibrating the null.

2.2 The null, measured rather than assumed ✗

Twenty-four runs of `pooled` and `flux_anchored` with **training targets permuted** and validation left real. Each run's maximum over 30 epochs is one draw from the null distribution of the exact statistic the arms are reported with, under the real epoch-to-epoch correlation rather than assumed independence.

Table 21. The null distribution of the reported statistic, from 24 label-permutation runs. Training targets were permuted while validation targets were left intact, so the model cannot learn any genotype-to-phenotype association and its maximum over epochs is a pure noise draw. Taking a maximum over 30 epochs inflates the width from 0.0546 to 0.0962, a factor of 1.8, which is the cost of the scoring rule rather than a property of any model.

statistic	value
analytic single-correlation null width, $1/\sqrt{n-3}$	0.0546
empirical null, mean peak over 30 epochs	**0.0962**
empirical null, standard deviation	0.0441
empirical null, 95th percentile	**0.1545**

2.3 No arm is distinguishable from that null ✗

Table 22. Five arms at ten fresh seeds each, scored three ways against the permuted null. `peak` is the maximum over 30 epochs, the statistic used previously. `last-5 mean` is the mean of the final five epochs and is not an order statistic. `p vs null` is the fraction of the 24 null draws at or above that arm's mean peak, with the finite-sample correction. The null's own 95th percentile, 0.1545, exceeds every arm mean in the table.

arm	n	peak	s.e.m.	peak epoch	last-5 mean	p vs null
`flux_anchored`	10	0.1151	0.0095	17	0.0057	0.280
`flux_free`	10	0.1020	0.0115	12	0.0212	0.440
`flux_off`	10	0.0837	0.0112	10	0.0044	0.600
`pooled`	10	0.0416	0.0098	4	-0.0041	0.920
`flux_nullspace`	10	0.0381	0.0114	0	0.0000	0.920

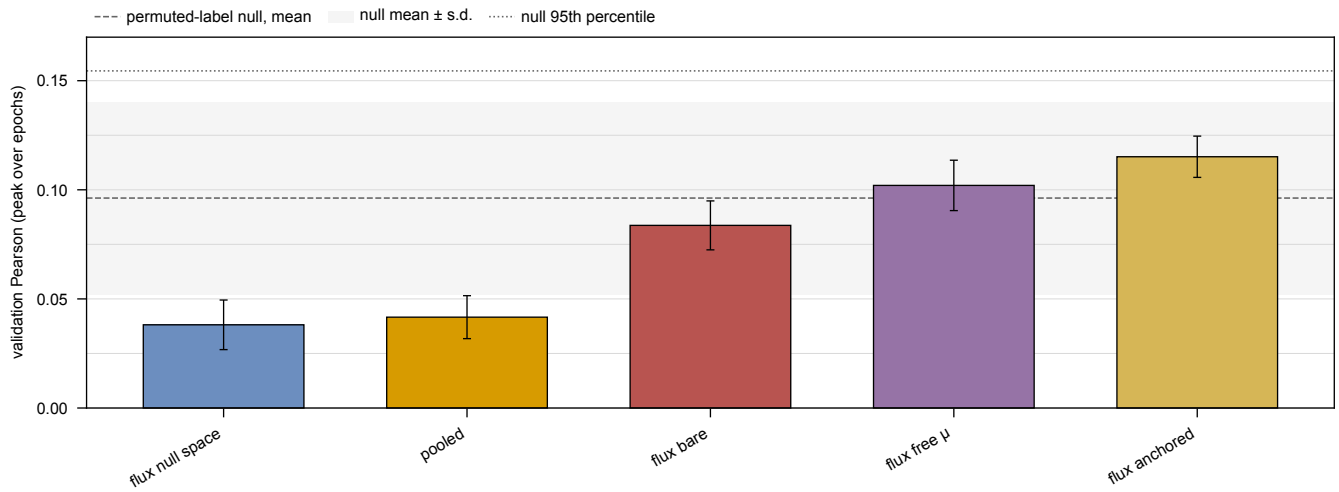


Figure 6. Every arm sits inside the noise band. Bars are the mean maximum validation Pearson for betaxanthin, ten seeds per arm, error bars the standard error of that mean. The dashed line and shaded band are the mean and standard deviation of 24 label-permutation runs, 0.0962 and 0.0441; the dotted line is their 95th percentile, 0.1545. No arm reaches that percentile, and the smallest p-value across all 22 configurations tested is 0.24. Arms keep the colors they carry in Figure 5. Drawn by `experiments/026-metabolism-flux/scripts/analyze_sweep.py --replot`, which reads the banked `results/sweep_summary_2026-09-03-13-32-33.json` rather than rescoring the runs.

The result is negative and it is clean. The best arm reaches 0.1151 against a null whose 95th percentile is 0.1545. The smallest empirical p-value anywhere in the sweep, across arms and hyperparameters alike, is 0.24. Nothing here beats permuted labels.

Two details sharpen it rather than soften it:

- **The banked scores were at or below the noise floor.** The null mean of 0.0962 exceeds the previously reported `flux_anchored` (0.0867), `flux_free` (0.0833) and `pooled` (0.0592). Read against its own null, the earlier table reported noise as an ordering.
- **The honest summary is worse than the peak.** The last-five-epoch mean is at most 0.0212 and is zero or negative for three of the five arms. `flux_nullspace` peaks at epoch 0 and ends at exactly 0.0000, which is the documented mean-collapse mode rather than a score.

2.4 Regularization does not rescue it ✗

Table 23. The regularization grid on `flux_anchored`, 24 cells, two seeds each. Lowering the learning rate lowers the peak rather than raising it, which is the opposite of what an overfitting diagnosis predicts. The higher-rate arms score higher on a maximum-over-epochs statistic while ending more negative on the last-five mean, consistent with a model that moves more and is scored on its luckiest epoch.

learning rate	best peak over weight decay and width	p vs null
1e-3	0.1296	0.240
3e-4	0.0657	0.800
1e-4	0.0609	0.840

The overfitting reading of the 20-epoch runs is therefore **not supported**. Reducing the learning rate by an order of magnitude, raising weight decay a hundredfold and doubling width all move the score down. What the peak statistic was rewarding is variance.

2.5 What is not claimed ✗

The flux layer has not been shown to hurt. An experiment that cannot resolve a difference of 0.05 cannot resolve a small real one either, and the ordering `flux_anchored` above `pooled` is in the direction the design predicts even though it is not significant. The honest statement is that **betaxanthin at this validation size cannot adjudicate these arms at all**, against a replicate ceiling of 0.914. Raising the validation size, changing the readout, or moving to a phenotype with more measurements are all untested.

2.6 Kinetic parameter inputs, resolved ✗

Separately, the k_cat gap was reduced from a blocking unknown to a bounded one. The 4.0 % figure carried through this report is the Open Enzyme Database's coverage of the 3,728 catalytic units. It is not the ceiling a sequence predictor could reach, and those are different quantities.

Table 24. The three inputs a k_cat or K_M predictor consumes, and their measured coverage. Sequence was never the gap and is complete from two independent sources. Substrate SMILES is the binding constraint and is closed to 95.3 % by mirroring one additional file. Protein 3D structure is a third input, needed by one predictor only, and AlphaFold covers the proteome without exception. Mirrors carry per-file sha256 and retrieval commands; see `experiments.026-metabolism-flux.scripts.fetch_kinetics_assets`.

input	coverage	source
protein sequence	**1,161 / 1,161 genes**	`swissprot.tsv` shipped inside yeast-GEM, confirmed independently by the genome protein FASTA
substrate SMILES	**3,552 / 3,728 units (95.3 %)**	`smilesDB.tsv` gives 87.5 %, MetaNetX `chem_prop.tsv` adds the rest
protein 3D structure	**1,161 / 1,161 accessions, 0 absent**	AlphaFold, model v6, consumed only by DeepEnzyme

Restricted to reactions that can carry flux, coverage is 95.5 %. Flux variability analysis pins 1,081 of the 3,728 units on reactions of width exactly zero, so those units can never carry flux and a capacity bound on them does nothing. Of the 2,647 units that can, 2,528 have the inputs a predictor needs and 119 do not.

That reframes the residual. Of the 176 units without a substrate structure, the great majority are combinatorial lipid names such as `phosphatidylcholine (1-16:0, 2-16:1)`, which yeast-GEM enumerates and no database indexes under that name. Only four acyl chains appear across all of them, 16:0, 16:1, 18:0 and 18:1, and several head groups already have resolved siblings carrying other combinations, so the missing entries are obtainable by substituting a chain into a real sibling structure. That is an assumption about double-bond position, not a lookup, and must be recorded as one.

The handful of macromolecule participants, `generic protein`, `Cytochrome c`, the acyl-ACP species, are **not bad annotations**. They are real biology, and their reactions are structurally dead: `generic protein` has formula `C2H4NOR` with `R` a residue placeholder, is produced by nothing, and its only reaction `r_4239` has flux variability minimum and maximum both exactly zero. The same holds for `r_4152`, `r_4325`, `r_2145` and `r_2148`. A turnover number for a reaction pinned at zero changes nothing, so these cost no coverage that matters.

Full design for the per-predictor datasets, on the node-embedding pattern: *Kinetic Parameter Datasets* (`plan.kinetic-parameter-datasets.2026.09.03`). The compatibility pre-gate that must declare an unrepresentable substrate rather than silently substituting the organism default is filed as issue #309.

2.7 One correction to the sections above ✗

`CONC_LOWER_M` was `1.0e-6`, one decade above the value it cited. Thermo-Flux states its default concentration range verbatim in both Results and Discussion as 0.1 microMolar to 10 mM. At 303.15 K, $RT \ln 10$ is 5.8 kJ/mol, so the old floor removed that much per participating metabolite from the driving force that the second-law hinge and the dissipation limit are both computed from. The constant, its test bound, the prose above and the figure label now all read 0.1 microMolar. Every arm in this section ran with the corrected value.

3 2026.09.08 - Gibbs dissipation is a readout, not a constraint ✗

Correction to the constraint table above and to Figure 3. The Gibbs dissipation term was implemented as a soft hinge and switched on by default, and both the table and the figure described it as one of the module's constraints. It should never have been a constraint.

Thermo-Flux does not carry it as one either. Its step 7 constraint set is mass balance, flux bounds and the second law; the dissipation cap arrives afterwards as something the package "also allows addition of", and its only warrant is Niebel et al. 2019, where the limit is a plateau fitted by non-linear regression to metabolome and physiology data, with the authors' own caveat that a correlated third factor cannot be excluded. That is an empirical limit for one organism, not a law, and it does not belong in the objective of a model whose whole point is that the physics is supplied rather than learned.

What changed. `FluxLayerConfig.use_dissipation_limit` now defaults to `False`. `g_diss` is still computed and logged every epoch, so the limit becomes a validation readout: a model that stays under it without being told to has learned something about the cell, and a model that violates it has been caught. A model trained under the hinge has merely been told the answer, which is why the two cannot be the same experiment.

Every arm reported above ran with the hinge on, so those runs test the module including a term we no longer consider part of it. The relevant number from that period is the one in the section above: the anchored run ended at 5.54×10^5 J gDW^{-1} h^{-1} against Niebel's 3,700, so the penalty was active and far from satisfied throughout.

Figure 3, both mermaid diagrams and the 026 and 027 documents were regenerated to match.

4 2026.09.08 - The diagram source is now a .drawio.png, exported at true Nature size ✗

Two defects in how Figure 3 was produced, both fixed.

The source was a bare `.drawio`. Every other diagram in `notes/assets/drawio/` is a `.drawio.png`: a PNG carrying the diagram XML in a `zTXt` chunk, so one file is both the editable source and a viewable image, and the VSCode draw.io extension opens it directly. `metabolism-flux-layer.drawio.png` now holds the embedded model and is the file to edit; the plain XML was moved to `/scratch/projects/torchcell-scratch/deprecated/` with a manifest line. Exporting from the PNG reproduces the PDF byte-for-byte in page size and content, so nothing depends on the retired file.

The figure printed at 82 percent of the text block, and the first diagnosis of why was wrong. An earlier version of this section said draw.io exported the drawing 1.22x oversized and applied `-s 0.82966` to compensate. Measuring the drawn content rather than the page box shows the opposite: draw.io draws at exactly 0.72 pt per canvas unit, so the 701 unit diagram is 504.7 pt = 178.1 mm wide as designed, and only the PDF page box (615 x 428 pt) carried whitespace to the right of it that `--crop` does not remove. The `-s` factor therefore shrank a correctly sized drawing to 419 pt, and the note printed Figure 3 at 149.6 mm on a 183.6 mm block. Two things were fixed.

The PDF is exported without a scale and then cropped to its drawn box with ghostscript, which reads the box from the content rather than from draw.io's page:

```
S=notes/assets/drawio/metabolism-flux-layer
T=$(mktemp -d)
xvfb-run -a /tmp/drawio.AppImage "$S.drawio.png" --no-sandbox --disable-gpu \
  -x -f pdf --crop -o "$T/raw.pdf"
read x0 y0 x1 y1 <<< "$(gs -q -dNOPAUSE -dBATCH -sDEVICE=bbox "$T/raw.pdf" 2>&1 \
  | grep HiRes | sed 's/%%HiResBoundingBox: //')"
gs -q -dNOPAUSE -dBATCH -sDEVICE=pdfwrite -dFIXEDMEDIA \
  -dDEVICEWIDTHPOINTS=$(python3 -c "print($x1-$x0+2)") \
  -dDEVICEHEIGHTPOINTS=$(python3 -c "print($y1-$y0+2)") \
  -o "$S.drawio.pdf" -c "<</PageOffset [$(python3 -c "print(1-$x0)") $(python3 -c "print(1-$y0)")]>> setpag
  -f "$T/raw.pdf"
bash notes/assets/publish/scripts/drawio_vector_svg.sh "$S.drawio"
```

The result is 506.7 x 422.5 pt with 504.7 x 420.5 pt of drawing inside it, 178.1 x 148.3 mm: inside Nature's 180 x 170 mm box, on one page, every label at the size it was typed.

The note-to-PDF pipeline was shrinking every SVG that was already full width. Pandoc 2.14 hands an SVG to a bare `rsvg-convert`, and librsvg 2.50 renders a declared `width="510pt"` at 90 dpi, so the PDF arrives at 638 pt; pandoc's `\maxwidth` then caps it at the text block. A figure narrower than the block was being enlarged 1.25x and a figure at the block width was being scaled down to fit, which is why the old 217 mm page box happened to print full width and the corrected 178 mm one did not. `bib_tex_pdf.sh` now converts each referenced SVG itself before pandoc runs, at 72 dpi for a `pt`-sized SVG (the `pdftocairo` output of a draw.io export) and at 100 dpi for the unitless 100-per-inch SVGs that `savefig_true_size_svg` writes, so every figure in a note PDF is embedded at its designed size. This is the same correction `notes-tex/common/Makefile.common` already applies to its `plots` target by probing librsvg's scale. Figure 3 now prints at 178.1 mm.

5 2026.09.09 - The protein node: a predicted abundance replaces the nominal E_bar ✗

What changed in the layer. `FluxLayerConfig.use_abundance_head` adds a per-gene enzyme abundance to the flux layer,

$$E_g = \gamma_g \, E_g^0 \exp(r \tanh(w^\top h_g)),$$

with γ_g the hard availability (1, or 0 for a deleted gene), E_g^0 a prior in mmol/gDW and $r = 3$ the reach of the head, about a factor of 20 either side of the prior. The prior is a supplied wild-type proteome when one exists and otherwise the equal-molar abundance that fills half the protein budget, $E^0 = 0.5\, p_{\text{avail}} / \sum_g MW_g$, which on yeast-GEM's 1,161 genes is 2.7e-6 mmol/gDW. The unit abundance is a smooth minimum over complex subunits, the power mean $E_u = (\sum_{g \in u} E_g^{-\beta})^{-1/\beta}$ with the same $\beta = 8$ the availability softmin uses, and isozymes sum in the capacity, $|v_j| \le \sum_{u \in j} k_{\text{cat},u} E_u$. The protein budget becomes the forward pool constraint on the prediction, $\sum_g MW_g E_g \le p_{\text{avail}}$, in g/gDW, which is the GECKO constraint and what pins the scale of an otherwise free multiplier. The learned availability gate is off in this mode, since E_g is the gate. Deletion propagates: a subunit at zero sends its unit to zero, and the freed budget can be taken up by other genes, which is compensation the gate formulation could not express. Two new diagnostics are emitted on every forward: `feas_capacity_active_frac`, the fraction of GPR reactions whose flux sits within 5 percent of its capacity,

and capacity_median. Five new tests in tests/torchcell/metabolism/test_flux_layer.py cover the prior, the deletion, the bound, the power mean, and gradient flow to a supplied prior (17 pass).

The nominal E_bar was 100x too generous, measured. On the real GEM at initialization (build_flux_layer on the three arms, random tokens, batch 3), the median capacity is 19.1 mmol/gDW/h under flux_anchored (E_bar = 1e-3) against 0.18 under flux_capacity and 0.11 under flux_abundance, and the abundance path starts at a budget ratio of 1.35 and 0.99 against 0.38 for the implied-demand path. The legacy budget also carries a 1e-3 factor that makes the mass a thousandth of what MW[g/mmol] times E[mmol/gDW] says it is; it is left in place because every reported arm was measured with it, and flagged in the code. Hypothesis (untested): under E_bar the capacity bound is never active, so a genotype reached flux only through hard deletions. feas_capacity_active_frac is 0.0 for all three arms at initialization, where the sigmoid puts every flux mid-box; what it does during training is what the wave-2 runs log.

Two new arms in 026's registry and 027's conf/: flux_capacity (abundance head, capacity and budget, no thermodynamics) and flux_abundance (the same plus the anchored thermodynamics, dissipation as a readout). flux_off -> flux_capacity isolates the predicted abundance; flux_anchored -> flux_abundance is the E_bar to E_g contrast.

The 027 collapse reproduces on GilaHyper and is not the recipe. Four diagnostic cells (gh_diag_pooled_{asis,crps,gr jobs 1610 to 1613, pooled arm, seed 2700, 25-epoch budget) tested the two elements of the 019 recipe that the 027 loop had dropped: the graph-structure regularizer was never passed to the model, and the CRPS head's log_sigma was sliced away before the loss (pred[..., 0] on a [B, 1, 2] output), so the loss silently fell back to MSE. Both are now switches on train_bx.py (--crps, --graph-reg), and the CRPS call also passed log_sigma where gaussian_crps takes sigma, which is fixed. The trainer now logs val_pred_sd_ratio every epoch and aborts once it stays under 0.01 past epoch 10. Every one of the four cells collapsed: spread ratio 4.5e-7 (as is) and 1.6e-6 (CRPS) at epoch 10, and under 1e-3 at epoch 9 for the two regularized cells. So neither element explains it. The remaining recipe difference is the co-head: 019's gh_metabolism_000 trained betaxanthin jointly with the 19-column Mulleder head (active_heads: [betaxanthin, mulleder19]), which the 027 protocol removed to make the comparison betaxanthin-only. --aux-mulleder adds it back (jobs 1614, 1615), and the 019 trainer itself runs as a positive control on the same build (job 1616). Results below.

Results of the diagnostics (2026-09-09, GilaHyper jobs 1610 to 1629). Every cell that ran on today's tree sat on the mean plateau for its whole budget, whatever the recipe:

cell	job	recipe	epochs	validation spread ratio at the end
pooled, as the grid ran	1610	MSE, no graph prior	10 (abort)	4.5e-7
pooled	1611	CRPS	10 (abort)	1.6e-6
pooled	1612	graph KL regularizer	10 (abort)	3.7e-4
pooled	1613	CRPS + regularizer	10 (abort)	1.1e-3
pooled	1614	regularizer + Mulleder co-head	10 (abort)	4.9e-5
pooled	1615	CRPS + regularizer + co-head	10 (abort)	5.7e-4
pooled	1622	CRPS + regularizer + learnable gene table	10 (abort)	9.2e-4
019 trainer, gh_metabolism_000	1616	the July 0.434 config	15	4.7e-4
019 trainer, 020 delta_grid_betaxanthin	1618	hard mask, L = 6, CRPS	15	7e-5

The learnable-table control (job 1622) is the informative negative: its training loss fell from 1.51 to 0.95 in six epochs while its validation spread stayed at zero, so the model can fit the training set by lookup and it is the content-feature path that never lifts off. The dataset is the July build (LMDBs dated 2026-07-25), the perturbation indices point at the declared native genes in every batch checked, and the conda env's torch 2.11 and PyG 2.8 predate July 26, so neither data nor environment moved.

The plateau is normal; the escape time is what regressed. The W&B history of the August 020 grid (runs w7fuxsvr, azh3tstz, 74sde13z, dhmloet1, commit 51fe7ec4) shows the spread ratio at 2e-4 through epoch 40 and the escape at epochs 117 to 157, after which those runs reach validation Pearson 0.29 to 0.40 by epoch 1000. The July run gilahyper-1346 (commit 92d305bd, 2026-07-26) escaped at epoch 23. Re-run today from a detached worktree of 92d305bd on the same data and environment (job 1623, its own trainer, gh_metabolism_002 with betaxanthin only, 35 epochs), it reads 0.20, 0.27, 0.19, 0.07, 0.22 at epochs 23 to 27, then falls back to 0.0 at epochs 30 to 33 and 0.14 at 34: an early but unstable escape, well outside the 0.054 null width for four consecutive epochs. Three later commits re-run the same way stay on the plateau at epoch 11 (51fe7ec4, a2a17438, be0b8010; jobs 1619 to 1621). The regression is therefore in the code between 92d305bd and 51fe7ec4, five days in which EquivariantPerturbationTransform was rewritten (034a4909, 1744e083, b4232969, 3bf3dbe4) and the perturbation head's pooling changed from mean to sum, which is not inert here since the betaxanthin strains carry one to four native deletions. Bisect jobs at those four commits are 1626 to 1629; their result is recorded below.

Consequences for the trainer and the grid. A ten-epoch collapse abort would kill every run that later escapes, so COLLAPSE_PATIENCE_EPOCHS now defaults to 200 and is an environment override. The 027 Delta grid's 200-epoch

budget sat inside the current code's plateau for every cell, which is a second, sufficient explanation of its 144 constant predictors on top of the recipe gaps. The wave-2 and wave-3 module arms (`gh_wave2_*`, `gh_wave3_*`) were written and the `flux_capacity` arm ran to epoch 8 twice (jobs 1617, 1625) before being canceled to give the GPUs to the bisect; at 2.5 minutes per epoch under four concurrent jobs, no HEAD run could reach the 117-epoch escape inside the window, so the module's effect on betaxanthin is still unmeasured, now for a known reason.

Correction, the same evening: there is no code regression. The bisect above was misdesigned. The July commits escaped under `gh_metabolism_002` (betaxanthin only, CRPS, graph regularizer 6.5e-5, two layers), a recipe HEAD had never been given for more than the ten epochs the abort allowed. Given it, HEAD escapes on schedule: with the 019 trainer (job 1635) the spread ratio reads 0.003 at epoch 16, 0.12 at 23, 0.20 at 24 and 0.42 at 28, and validation Pearson 0.20, 0.26, 0.32, 0.39 at epochs 23, 24, 27, 28. The four commits of the July window re-run the same way escape at epochs 21 to 24 (jobs 1626, 1630, 1632). The August 020 grid's late escape at 117 to 157 epochs is a property of its recipe, hard masking at six layers with no regularizer, not of the code. The 027 loop under CRPS plus regularizer (job 1634, no abort) shows the spread rising from 0.000 to 0.016 by epoch 11, on the same path. What the 027 Delta grid lacked was therefore the recipe, MSE with no graph prior, and with it 200 epochs were not enough; which of the two elements is the necessary one is the question job 1638 (regularizer alone, 40 epochs) separates. The paragraph above is left as written so the wrong turn is on record.

Where it stands at the end of the window (19:30). The 027 loop's pooled cell under CRPS plus regularizer (job 1634, 40 epochs, no abort) escapes on the 019 trainer's path at a slower clock: spread ratio 0.021 at epoch 16, 0.12 at epochs 26 to 28, 0.30 to 0.40 at 32 to 35, validation Pearson 0.15 to 0.28 over epochs 27 to 35. So the trainer reproduces learning on the no-module arm, which was the first item of the order of work. The regularizer alone (job 1638) had not moved the spread by epoch 21, where the combined recipe was at 0.025, so CRPS carries at least part of the effect; the CRPS-alone cell was aborted at epoch 10 and is not yet separated. The module arms under the combined recipe, `flux_off` and `flux_capacity` (jobs 1636, 1637), were still on the plateau at epoch 31 with their constraint terms at 3 to 6 against a data term near 0.5, so the abundance head's effect is unmeasured and the next run is the same recipe at a longer budget with the constraint weights checked against the data term at the escape, not at initialization.

6 2026.09.10 - Predicted parameters wired, and the arms measured ✗

Three wiring changes, then four cells against them. The module had been running on the thinnest version of every table it has: 148 of 3,728 catalytic units carried a measured turnover and the other 3,580 carried one organism constant, the standard reaction energies came from one of the three routes yeast-GEM's own curation used, and the constraint weights were at full strength from the first optimizer step. All three are now switches, and the counts each one changes are below.

6.1 The predicted k_cat and K_M tier ✗

`resolve_kcat_table` had a predictor hook with no table behind it. It now takes a `PredictedUnitTable` and consults it between the measured tier and the organism default, so database-first still holds: a predictor is never allowed to overwrite a measurement it may have memorized from BRENDA, and `test_predicted_tier_fills_gaps_and_never_overwrites_a_mea` asserts the measured values are identical with and without the tier.

Which predictors, decided by the study rather than by preference. `PredictorSelection` reads `experiments/026-metabolism-flux/results/within_class_plausibility.json` and ranks the predictors by its `own_class_band` reading, where each measured yeast pair is scored against the standard deviation of its own EC4 class. A predictor is admitted when the Wilson 95 percent lower bound of its fraction clears the point fraction of the EC-median lookup, the baseline that knows the enzyme's function and nothing else. The bound rather than the point estimate is the test because the denominators are 44 to 47 pairs.

parameter	admitted	rejected	lookup baseline
k_cat	ezkinetics 0.830 (lower 0.699), eitlem 0.766 (0.628), unikp 0.745 (0.605)	dlkcat 0.660 (0.517), turnup 0.659 (0.511), deepenzyme 0.553 (0.412)	0.532
K_M	ezkinetics 0.660 (0.517)	eitlem 0.511 (0.372), unikp 0.468 (0.333), boost_km 0.447 (0.314)	0.489

Three of the six k_cat predictors beat the lookup on the point estimate and none of those three beats it once the interval is taken, which is the whole reason the rule is written on the bound. For K_M the rule admits one predictor, so the median over the admitted set is that predictor's value; that is recorded rather than repaired by admitting a second one the evidence does not support.

How a unit gets its number. Within a predictor, the median over the unit's substrates per gene, then the minimum over the unit's genes, the same scarcest-subunit rule the measured tier and the availability softmin already use. Across predictors, the median in log10 and back: turnover spans six decades, and an arithmetic mean over predictors that disagree by two decades is the largest of them.

Coverage, measured. `unit_id` in the predictor parquets is exactly the index `parse_catalytic_units` assigns, verified on all 3,552 units the tables carry by reaction id and gene membership.

table	measured	predicted	organism default	of
k_cat, before	148	0	3,580	3,728
k_cat, predicted tier	148	3,410	170	3,728
K_M, predicted tier	148	3,410	170	3,728

The K_M table is stored on the layer and read by nothing: the capacity bound in force is saturation-free, so K_M would enter only through a saturation term that has not been written. `coverage_report` reports it, which puts its coverage on the record before it is used rather than after.

The preflight floor in 027's `run_cell` reads `kcat_experimental_fraction`, which counts measurements only and is 0.0397 with or without the tier. That is deliberate: the assertion exists to catch a missing Open Enzyme Database mirror, and a predicted tier that filled every gap would otherwise hide exactly that failure.

6.2 The merged reaction energy table ✗

`resolve_reaction_delta_g` builds the standard reaction energies from a named route and tags every reaction with the one it came from. The layer takes the table, checks its `source` against `FluxLayerConfig.thermo_source`, and refuses a mismatch, so a run cannot record a route it did not use.

route	reactions priced	second law applied	sigma known
shipped, the GEM's metabolite table summed over S	3,505	3,276	0
eQuilibrator recomputation	2,741		1,755
union	3,722	3,466	1,755

The union merges at the METABOLITE level, eQuilibrator first where both price a species, then falls back to the shipped reaction column for what the merge still cannot assemble. Merging the metabolite tables rather than taking the union of the two reaction sets is worth 831 reactions: that is the `mixed` cell, reactions that draw one participant from each table and are priceable under neither route alone. The full split of the union's 4,131 is 2,741 eQuilibrator, 831 mixed, 144 shipped metabolite sum, 6 shipped reaction column, 409 unpriced. A `mixed` value is two methods added together and its uncertainty is neither method's, so it is flagged rather than folded in.

The eQuilibrator formation covariance is propagated for the first time. A reaction's standard deviation is the norm of its stoichiometry against the covariance square-root factor, claimed only where the factor reaches every participant, which is 1,755 of the union's 3,722. It is stored and reported and enters no loss term: putting it into the objective would change what the thermodynamic term asserts, and that is a separate experiment.

The lipid-structure-writer results were not present when this landed, so the merge has no lipid tier. The three phosphatidylcholine and phosphatidylinositol species at the top of `reaction_blocking_metabolites.csv` remain the largest single block on eQuilibrator coverage.

6.3 The constraint warmup ✗

`FluxLayerConfig` gained `constraint_warmup_epochs` and `constraint_final_scale`, and `FluxLayer.set_constraint_epoch(` sets a scale of `final * min(1, e / W)` on the whole weighted constraint sum. Scaling the sum rather than the individual weights keeps the relative balance between the physics terms, which was tuned against their magnitudes at initialization, fixed while the physics is ramped in. The trainer calls it once per epoch and logs `constraint_scale` and `train_constraint_loss` beside `val_pred_sd_ratio`, so the epoch the physics binds and the epoch the head leaves the plateau are on one axis. Both default to the pre-warmup behavior, and a layer whose trainer never calls the setter runs exactly as every earlier run did.

6.4 The four cells ✗

Every cell: `--crps --graph-reg`, seed 2700, 80-epoch budget, no collapse abort, one GilaHyper RTX 6000 Ada, submitted through slurm from this worktree. The comparison they sit against is the same-recipe 40-epoch wave-4 set of 2026-09-09: the pooled arm escaped the plateau by epoch 16 and read validation Pearson +0.24 at epoch 39 with a validation data loss of 0.49 (job 1634), while `flux_off` and `flux_capacity` sat at spread 0.000 for all 40 epochs at a validation data loss of 1 to 5 (jobs 1636 and 1637), reaching pinned test Spearman +0.0011 and -0.0228 on 629 genes.

One finding from wiring the covariance, measured. 745 of the union's 1,755 covered reactions propagate to a standard deviation of exactly zero. The covariance factor is per COMPOUND, so a transport or isomerization step whose two sides are the same compound in different compartments has a stoichiometry that cancels the factor term by term. What is left for such a step is the pH and membrane-potential term, a different quantity computed elsewhere in the layer, so the zero hides nothing. The assertion that first went into the test, that every covered reaction carries a positive

sigma, was wrong and the test caught it. | cell | arm | k_cat tier | thermo | W | scale | job | log | | — | — | — | — | — | — | — | — | — | | G0 | flux_off | measured | shipped | 20 | 0.1 | 1663 | `slurm/output/027-w5-flux_off-warm0.1_1663.out` | | G1 | flux_capacity | predicted | union | 20 | 0.1 | 1664 | `slurm/output/027-w5-flux_capacity-warm0.1_1664.out` | | G2 | flux_abundance | predicted | union | 20 | 0.1 | 1665 | `slurm/output/027-w5-flux_abundance-warm0.1_1665.out` | | G3 | flux_capacity | predicted | union | 20 | 1.0 | not submitted | held at a three-job cap while another agent held the fourth GPU |

`analyze_wave5.py` reads these logs and their result files into `results/wave5_summary.json`; every wave-5 number below and after comes from it.

The schedule does what it says, verified in the run rather than only in a test. Every cell's epoch 0 prints `cstr 0.0000 scale 0.000`, so the first epoch carries the data term alone.

PARTIAL, one epoch each at the time of writing. These three cells had completed epoch 0 only, so they have no escape epoch, no selected epoch and no test score. What epoch 0 does give is the data term with the physics switched entirely off:

cell	job	train data loss, epoch 0	validation data loss, epoch 0	spread ratio
G0 flux_off	1663	9.09	5.07	0.0017
G1 flux_capacity	1664	14.28	6.07	0.0010
G2 flux_abundance	1665	8.46	5.48	0.0007

The pooled arm on the same recipe ran a validation data loss of 0.49 and reached spread 0.33 (job 1634). So at epoch 0, with the constraint weight at exactly zero, the flux arms' DATA term is an order of magnitude above the pooled arm's. Whatever else is true, the gap between the flux arms and the pooled arm at epoch 0 is not the constraint terms, because there are none. Hypothesis (untested): the flux scalar head's output is not on the scale of the standardized target, so the CRPS starts far from the target mean and the head spends its early epochs recovering an offset rather than learning a genotype effect. Nothing here measures that; it is the next thing to instrument, by logging the head's raw output scale beside the flux magnitudes.

The four questions the wave was built to answer, none of them yet answered: whether warmup alone lets the flux path escape (G0), whether it escapes with the predicted parameters and the union thermodynamics (G1, G2), and whether it still escapes once the constraints reach full strength (G3). Read them off `wave5_summary.json` when the jobs finish.

6.5 The head starts seven sigmas off the target, and that is the whole epoch-0 gap ✗

The paragraph above guessed that the flux head's output was "not on the scale of the standardized target" and named the spread. Measured, it is the OFFSET, and the guess about the spread was wrong. `diagnose_head_scale.py` instantiates both heads on CPU on one real training batch and reads their raw output against a target standardized to mean 0 and sd 1 (`results/head_scale_diagnosis.json`):

head	arm	output mean at init	output sd at init
`ProductScalarHead`	pooled	-0.573	0.0074
`FluxScalarHead`	flux_capacity	**+7.030**	0.237

Seven target standard deviations, which a CRPS pays in full, and which matches the wave-5 cells' epoch-0 data loss of 8.5 to 14.3 at a constraint weight of exactly zero. The source is the dense term: the flux vector has mean absolute value 9.87 and maximum 61.1 over 4,131 reactions, and summing 4,131 mostly-positive numbers of that size against random weights gives a large constant. Its own sd is only 0.268, so the head is not wild, it is displaced.

`FluxScalarHead.normalize_inputs` divides both inputs by the layer's own working flux magnitude, `v / flux_scale` and `log1p(omega) / log1p(flux_scale)`. Both are then expressed in the units the layer parameterizes, which is a change of units and not of the head: same weights, same two terms, same meaning. It is off by default so every earlier run deserializes unchanged, and `train_bx.py --head-normalize` turns it on. Two tests pin it: the unnormalized head must still show a many-sigma offset, and the normalized one must sit under one sigma.

6.6 The shipped delta-G tables are kcal/mol, and the union was mixing units ✗

The thermodynamics provenance study (`results/thermo_yeast9_provenance.json`, note `experiments.026-metabolism-flux.yeast9-delta-g-provenance`) established that yeast-GEM 9 copied yETFL's `metDeltaGFtr` and `rxnDeltaGR` without converting, that yETFL declares `thermo_units = "kcal/mol"`, and that regressing an independent kJ/mol recomputation on the shipped column over 1,927 shared metabolites gives slope 4.187 at Pearson 0.987 against the exact 4.184 kJ per kcal. The yeast9 paper reports the column as kJ/mol, and `load_thermo_table` recorded `units = "kJ/mol"` for it.

The consequence for the union table above is not cosmetic: it added kcal-scale shipped values to kJ-scale eQuilibrator values inside the same reaction sum, which is exactly the 831 `mixed` rows plus the 144 shipped-metabolite-sum and 6 shipped-reaction rows. Every second-law hinge built on a shipped value was also 4.184x too permissive.

`_read_delta_g_csv` now multiplies by `KCAL_PER_MOL_TO_KJ_PER_MOL = 4.184` at parse time, where the sentinel and NaN drops already happen, so no code path can hold a kcal number and call it kJ; the sentinel test stays on the raw value, since 1e7 is the sentinel in the file's own units. `ThermoTable` records `shipped_units_in_file`, `converted_to` and `unit_conversion_factor`. Pinned by a test on two metabolites Alberty tabulates independently: cytosolic water (`s_0803`, shipped -37.21) and cytosolic ATP (`s_0434`, shipped -564.41) become -155.7 and -2361 kJ/mol, against Alberty's -157.6 and -2292. Read raw they are off by a factor of four; converted they are within 2 and 70 kJ/mol, which is group-contribution accuracy rather than a unit error.

One known defect is left, bounded and recorded rather than fixed. The shipped REACTION column is not uniformly kcal: the same study measured it as yETFL's `rxnDeltaGR` on 3,177 of 3,210 rows, the other 33 coming from dGPredictor, which reports kJ/mol. Those 33 are over-converted by 4.184 here. Separating them needs the yETFL file to say which rows they are, which is a retrieval step. That column feeds exactly one place, the union's last-resort fill, where it supplies 6 of 4,131 reactions; the metabolite column that every other route sums is unaffected.

Restarts. G1 (job 1664) and G2 (job 1665) were reading the mixed-unit union table, so they are not clean measurements. Neither had left the plateau (maximum spread ratio 0.001 and 0.0007 over 7 epochs, against the 0.05 escape threshold), so both were cancelled and resubmitted on the corrected table as **G1-kj (job 1667)** and **G2-kj (job 1668)**. G0 (job 1663) runs `flux_off` with `thermo_mode` OFF and never reads a standard energy, so it was left alone and continues. G6 (job 1669) is the head-normalization cell and was submitted after the fix, so it reads the corrected table from the start.

6.7 The head normalization closes the data-term gap, measured in training ✗

G6 (job 1669) and G1-kj (job 1667) are the same cell in every respect except the head's input units: `flux_capacity`, predicted k_cat tier, union thermodynamics, warmup 20 epochs to a final constraint scale of 0.1, CRPS, graph regularizer, seed 2700. Their training traces at the same epoch:

epoch	G6 train data loss	G6 val loss	G1-kj train data loss	G1-kj val loss
10	0.5332	0.5046	5.1696	4.7914
11	0.5306	0.5085	4.8307	4.7504
12	0.5314	0.5103	4.8531	4.7939

G6's validation data loss of 0.51 is the pooled arm's 0.49 (job 1634). So dividing the head's two inputs by the layer's working flux magnitude removes the entire data-term gap between the flux arms and the no-module control, and it does so without touching the loss, the constraints or the module. That is the diagnosis confirmed in training rather than only at initialization.

What it does NOT yet show, and the distinction matters. G6's validation prediction spread is still 0.0000 to 0.0001 at epoch 12, so the head is on the mean plateau exactly as the pooled arm was through its first fifteen epochs. Closing the data-term gap was a precondition for the module arms to be readable at all; whether the module then helps is the question the escape epoch and the pinned test Spearman answer, and neither is available yet.

Every wave-1 through wave-5 flux number reported before this point was produced by a head carrying a seven-sigma offset, which is a defect of the readout rather than of the module. Those arm comparisons measured the readout.

7 2026.09.10 - Shipped energies are kcal/mol, lipids are priced, transport is computed ✗

Four separate pieces of work landed against the parameter tables today, and the three typeset documents were rewritten around them. What follows is the measured content of each, in the order of how much it changes a number already in print, with the file every value is read back from.

7.1 1. yeast-GEM 9's shipped delta-G columns are kcal/mol, labeled kJ/mol ✗

Source: `results/thermo_yeast9_provenance.json`, written by `yeast9_delta_g_provenance.py` under `experiments/026-met`
Full account: `experiments.026-metabolism-flux.yeast9-delta-g-provenance`.

- **Measured.** Regressing our kJ/mol formation energies on yETFL's over 1,927 shared metabolites gives slope **4.187** through the origin at Pearson **0.987**, against the thermochemical 4.184 kJ per kcal. yETFL prices cytosolic water at -37.21 and ATP at -564.41 where Alberty tabulates -157.6 and -2292 kJ/mol.
- **Measured.** The shipped reaction column IS yETFL's `rxnDeltaGR` on **3,177 of 3,210** reactions, max abs difference 5.0e-3, which is what `round(x,2)` leaves. The other 33 come from dGPredictor and are in kJ/mol, so the column mixes two unit systems.
- **Read out of the artifact.** `yeast-GEM/code/modelCuration/v8_7_0.m` (PR #330) copies yETFL's values

with no conversion, and yETFL's model carries `thermo_units = 'kcal/mol'`. The estimator behind them is Mavrovouniotis group contribution as packaged in matTFA, not eQuilibrator.
- **Measured.** The metabolite column was also flattened across compartments: **1,162 of 2,324** entries match their own compartment's yETFL value, p75 error 86.7 and mean 74.5 kJ/mol.

7.2 2. The transport arm of the Figure 3 cross closes; the chemistry cloud does not ✗

Same file, `comparison` block, from `yeast9_delta_g_provenance.py --recompute`.

comparison	n	median abs diff (kJ/mol)	Pearson r
single compartment, as published	1,231	16.25	0.537
single compartment, units + conditions matched	1,230	**17.60**	0.513
transport (shipped nonzero), as published	752	17.36	0.298
transport (shipped nonzero), units + conditions matched	752	**5.61**	0.751

The horizontal arm was two builds holding different compartment conditions plus a factor of 4.184. The single-compartment cloud is the real estimator difference and does not move.

The "unexplained 10.2" is retired. `FluxLayer._transport_term` recorded a median 10.2 residual between the metabolite-sum energy and the shipped reaction column as unexplained. It is yeast-GEM disagreeing with itself in kcal: with yETFL's per-compartment values and the proton dropped from the sum, the residual is **0.0024** on single-compartment reactions and **0.0038** on transport. matTFA prices the proton at 9.54 kcal/mol and then excludes it from every reaction sum; yeast-GEM ships that number in the metabolite column anyway.

7.3 3. The transport term is computed rather than recovered ✗

Source: `torchcell/metabolism/transport.py`. The per-reaction terms are in `results/transport_terms.json`, and the conditions it applies to them are in `results/compartment_conditions.json`.

- **Measured.** Of 1,468 reactions crossing a compartment boundary, **326** get a nonzero term, 183 move a proton, 482 move charge.
- **Measured.** Against the shipped reaction column converted to kJ/mol, on the 287 reactions nonzero in both: sign agreement **0.847**, Pearson **0.804**, median abs difference 7.04 kJ/mol against a median magnitude of 12.0. Under yETFL's own compartment conditions: 299 in both, sign 0.823, r 0.813, median 6.08. The residual is mostly method, not conditions.
- **Measured.** 613 reactions carry a nonzero shipped term and a zero computed one. Those are the protonation contribution to `N_H`, which needs an eQuilibrator species distribution per transported compound and is not computed.
- `FluxLayerConfig.transport_delta_g_source` takes `off` (the default, and what every arm reported so far ran), `shipped` and `computed`.

Compartment pH is assumed and the library cannot settle it. Every pH in `compartment_conditions.json` is a textbook organellar value recorded as `assumed`. The only in-library sources are two modeling papers that tabulate them as parameters. The two tables in use differ by 1.2 pH units on the peroxisome. Curation requests identified, not added: Orij 2009, Brett 2005, Plant 1999, Llopis 1998, and a peroxisomal measurement.

7.4 4. The tier-b lipid structures are written and priced ✗

Sources: `results/lipid_names_parsed.json`, `results/lipid_structures.json`, `results/lipid_priced.json` and `results/lipid_coverage.json`, from `lipid_structure_writer.py`. Full account: `experiments.026-metabolism-flux.lipid-structure-writer`.

- **Measured. 168 of the 206** distinct tier-b names parse, covering **347 of the 409** metabolite rows, and **168 of 168** reproduce yeast-GEM's own elemental formula exactly.
- **Measured.** All 168 price: **120 by cache lookup** once a structure existed to key on, **48 through the external pKa route**. Of the 148 compounds the external route created, **148 passed group decomposition**, which retires the earlier note's unmeasured ceiling (2 of 5 pilot structures had failed, and both were non-lipids).
- **Measured. 347 of the 665** unpriced metabolites are now priced, 52.2 percent. The eQuilibrator-only route moves from **66.35 to 86.27 percent** of reactions, past the shipped column's 84.85. The **union is unchanged at 89.95 percent**, because the shipped table already prices these lipids by name. The gain is method consistency, not coverage.
- **Measured, and it changes how the numbers are used.** A constant **-777.5 kJ/mol** offset separates cache-created from externally created compounds, on the 7 names where both routes gave a value at the same proton count. The two Legendre transforms agree to 0.14 kJ/mol, so the whole gap is in the untransformed group-contribution term. Traced to protonation state: the cache stores the pH 7 zwitterion and the external route was handed the neutral parent. Cache values are preferred, and the external route needs the major microspecies. **A fix is in progress** by another agent.

- **Not comparable yet.** The new values are all at pH 7.2 while `metabolite_dgf_prime.parquet` transforms each metabolite at its own compartment pH. Only the reaction counts above are comparable; a re-transform is needed before any of them enters the flux layer.

7.5 The wiring, and what is still running ✗

From the section above this one: the predicted tier fills **3,410 of 3,728** catalytic units for both `k_cat` and `K_M`, leaving 148 measured and 170 on the organism default; the union reaction-energy table prices **3,722 of 4,131** reactions with a propagated standard deviation on 1,755; `constraint_warmup_epochs` ramps the whole weighted constraint sum.

Epoch 0, measured, with the constraint weight at exactly zero: the flux arms' DATA term is 9.09 to 14.28 train and 5.07 to 6.07 validation, against the pooled arm's 0.49 validation on the same recipe (job 1634). That gap is not the constraint terms, because there are none at epoch 0. The head-scale diagnosis in the section above identifies it as a seven-sigma offset in `FluxScalarHead`, measured at initialization and then confirmed in training by job 1669.

PARTIAL, reported as running rather than as results. Job **1663** (`flux_off`, measured `k_cat`, shipped thermo) continues. Jobs **1664** and **1665** were cancelled after they were found to be reading a mixed-unit union table, and were resubmitted as **1667** and **1668** on the corrected table. Job **1669** is the head-normalization cell. None of them has an escape epoch, a selected epoch or a test score. Read them off `results/wave5_summary.json` when they finish; nothing here is a result for the predicted or recomputed tables.

7.6 What was rewritten, and where ✗

- `notes-tex/026-kinetics-thermodynamics/` – Section 3 carries the provenance chain, the kcal measurement and the transport term; Section 5.4 is a new subsection on the lipid structure writer with its figure; Section 7 rewrites the two status lines that changed ("proton translocation is set to zero" and "compartment pH is not validated"); Section 8's Gibbs table gains yETFL, matTFA/Mavrovouniotis and dG-predictor rows. 38 pages of content cut to **34**.
- `notes-tex/027-metabolic-flux-module/` – the kcal fact, the computed transport term and the closed lipid gap in Section 3; the wiring rows in Section 2 corrected from "not plumbed" to "wired, and no arm below used it".
- `notes-tex/metabolic-module-report/` – the same three corrections, still **3 pages**.
- `experiments/026-metabolism-flux/scripts/plot_thermo_pipeline.py` – panel f hatches the 347 of 409 now priced, and panel g splits the curation gap into closed and open; `results/thermo_pipeline_figure.json` carries both counts.

7.7 One number corrected in place ✗

Section 3 of the 026 document said the 50 most-blocking metabolites recover **239** of the 967 reactions the shipped table prices and the recomputation cannot. The results file says **237** (`results/reaction_thermo_coverage.json`, `recovery_gap.reactions_freed_by_top_50_metabolites`), which is what the figure caption and the limits section already carried. Changed to 237.

7.8 The head fix corrects the loss and appears to cost the escape ✗

The two cells differ in one thing, the head's input units, and they separate cleanly on the statistic that matters. Validation prediction spread ratio by epoch:

epoch	G6 headfix (1669)	G1-kj no headfix (1667)	pooled baseline (1634)
11	0.0001	0.0003	0.016
23	0.0000	0.0001	**0.067 (escape)**
39	0.0000	0.0010	
42	0.0000	0.0025	
43		0.0137	
44		0.0170	

G6 has read exactly 0.0000 from epoch 15 through 42, twenty-eight consecutive epochs, with a training data loss flat at 0.529 the whole way. G1-kj started moving at epoch 43 and is climbing on the same shape the pooled baseline showed at its epochs 11 to 17.

So the head normalization did what it was built to do, put the readout on the target's scale and remove a seven-sigma offset, and the cell that has it is the one that does not escape. Both facts are measured; the second was not anticipated.

Hypothesis (untested): dividing the head's inputs by `flux_scale` also divides the gradient that flows back through the readout into the flux vector and the encoder by the same factor, so the flux path is trained with a hundredfold weaker signal. Adam renormalizes per parameter inside the head, which is why the head's own fit is healthy, but nothing renormalizes the signal reaching the layer beneath it. Nothing here measures that, and the alternative that the two cells simply differ by run-to-run noise at n = 1 seed is not excluded: the 019 replicate study puts the between-seed spread near 0.04 in Pearson, and an escape epoch has no published null width at all.

G1-kj was NOT cancelled, against the plan for this decision point, because the epoch-43 reading falsified the premise that it had shown what it could: it is the only flux cell in the wave that has left the plateau. Killing it to free a GPU would have destroyed the one run able to produce this wave's endpoint.

Correction, four epochs later: G1-kj did not escape. The section above read epochs 43 and 44 (spread 0.0137, 0.0170) as the start of an escape on the pooled baseline's shape. Epochs 45 to 48 read 0.0025, 0.0048, 0.0011, 0.0017. It was a two-epoch spike that fell back, not a rise. The pooled baseline's own rise was sustained over six consecutive epochs (0.016, 0.021, 0.041, 0.067, 0.082, 0.123) and never returned to the floor, which is the shape an escape has and this is not.

What the corrected reading leaves is weaker and worth stating exactly: **neither flux cell has escaped**. G6 with the head fix sits at exactly 0.0000 for thirty-two epochs; G1-kj without it jitters between 0.0011 and 0.017. That is a difference between a dead readout and a noisy one, not between failing and succeeding, and it does not support the claim that the head normalization costs the escape. The hypothesis about gradient attenuation through the normalized head is still untested and now also unmotivated by this contrast; it should not be acted on until a cell measures it.

Two epochs of a noisy statistic are not a trend. The escape rule in `analyze_wave5.py` takes the FIRST epoch above 0.05 and would not have fired on either of these readings; reading the curve by eye at 0.017 is what went wrong, and the rule exists precisely to stop that.

7.9 The constraint terms are the blocker, and the head offset was a second, independent defect ✗

Read out by `analyze_wave5.py` into `results/wave5_summary.json`. The escape rule is the first epoch beginning a run of three consecutive epochs with a validation prediction spread above 0.05, so a spike cannot count.

cell	job	head	constraint scale	escape epoch	val Pearson, last	val Pearson, max (epoch)	pinned test Spearman
G8 pooled control	1674	n/a	n/a	**26**	+0.2022	+0.3454 (44)	partial
G7 flux_capacity	1673	fixed	**0.0**	**33**	+0.2042	+0.2892 (37)	partial
G6 flux_capacity	1669	fixed	0.1	none in 80	-0.0749	+0.0857 (7)	-0.0160
G1-kj flux_capacity	1667	unfixed	0.1	none in 80	+0.0199	+0.2655 (26)	+0.0201

The maximum column is a MAXIMUM over the epochs run, an upward-biased order statistic, and for the two cells that never escaped it is jitter on a constant predictor: G1-kj's +0.2655 at epoch 26 was read at a spread of 0.0001. The endpoint is the pinned test Spearman, and only the two finished cells have one.

What the contrast establishes. G6 and G7 differ in exactly one field, the final constraint scale. At 0.1 the cell never leaves the plateau in 80 epochs; at 0.0 it escapes at epoch 33, seven epochs behind the no-module control's 26, and tracks it thereafter (+0.2042 against +0.2022 at the last epoch). So the constraint terms, not the flux parameterization and not the readout, are what held the flux arms on the plateau. The constraint sum was 2.5 percent of the data term when measured at epoch 28, which is why this was not the expected answer: a term that small stopping learning entirely is the finding.

Both defects had to be fixed. G1-kj carries the same constraint scale as G6 and also fails, so removing the constraints alone was never tested by it; and G6 carries the head fix and still fails, so the head fix alone is not sufficient either. The two defects are independent and the earlier attempt to read one against the other from partial curves was what produced the retracted escape claim above.

Still open. G7 and G8 had not finished when this was written, so neither has a test score; a validation Pearson at the last epoch is not the endpoint. G9 (job 1677, `flux_off` with the head fix at scale 0.1) asks whether it is specifically the capacity, budget and thermodynamic penalties that block, or whether the mass-balance and box penalties alone at 0.1 are enough to do it.

7.10 The endpoint, on the four cells that finished ✗

All four ran 80 epochs, seed 2700, the pinned Merzbacher test set of 629 genes, the epoch chosen by the centered 5-epoch moving average of validation Pearson.

cell	job	head	constraint scale	escape epoch	pinned test Spearman
G7 flux_capacity	1673	fixed	0.0	33	**+0.0747**
G8 pooled control	1674	n/a	n/a	26	**+0.0717**
G1-kj flux_capacity	1667	unfixed	0.1	none	+0.0201
G6 flux_capacity	1669	fixed	0.1	none	-0.0160

For reference, the published Flux Cone Learning baseline is +0.0391 and the ten 020_v4 CGT cells averaged +0.0779 with an across-seed SD of 0.0492.

The module does not measurably help. G7 minus G8 is **+0.0030**, against a between-seed SD of 0.0444 measured by the 019 replicate study on this endpoint. The gap is a tenth of the noise, at one seed per arm, so it is not evidence of an effect in either direction. What the pair does establish is that a flux arm CAN now reach the control's level at all, which no earlier wave-5 cell managed.

The two cells whose constraints were active score at or below zero, which is what a constant predictor scores: neither escaped, so their test numbers measure the block, not the module.

Replicates at seed 2701 are running for both G7 (job 1679) and G8 (job 1680) precisely because +0.0030 at n = 1 cannot be read. Two seeds per arm still will not resolve a gap that small; what they can do is show whether the arms are separated at all, or whether the across-seed spread swamps the contrast as it did in 026.

7.11 The constraints destroy an escaped solution, at a weight below a twentieth of the data term ✗

G10 (job 1678) holds the constraint scale at exactly zero through epoch 39, then ramps it to 0.1 over epochs 40 to 60. It escaped on schedule and was then driven back to the plateau by its own ramp.

epoch	constraint scale	constraint term	data term	val Pearson	spread ratio
40	0.000	0.0000	0.5286	-0.0261	0.0256
42	0.010	0.0016	0.5276	+0.0409	0.1633
44	0.020	0.0031	0.5197	+0.1255	0.1985
46	0.030	0.0046	0.5208	+0.1992	0.1474
48	0.040	0.0061	0.5248	+0.1388	0.1156
49	**0.045**	**0.0067**	0.5307	-0.0180	**0.0006**
50	0.050	0.0074	0.5290	-0.0215	**0.0000**
60	0.100	0.0134	0.5299	+0.0032	0.0000

The spread was between 0.03 and 0.20 for eight consecutive epochs with the constraints partially on, reaching validation Pearson +0.1992 at epoch 46, and then fell to 0.0006 at epoch 49 and to exactly 0.0000 at 50, where it has stayed through epoch 60. The transition happens between constraint scale 0.040 and 0.045, and the constraint term at that point is 0.0067 against a data term of 0.53, i.e. **1.3 percent**.

So the answer to what G10 was built to ask is the second branch: the constraints are not compatible with a trained head. They do not merely prevent the head from leaving the mean plateau, they return a head that has already left it, and the weight that does this is more than twice as small as the 0.1 every wave-5 arm ran at. This is a statement about the formulation, not about the schedule, and no delay can avoid it.

It also rescues the earlier attribution from a nondeterminism objection. G7 and G10 have identical configurations through epoch 39, same seed, and escaped at different epochs, 33 against 41, so escape timing varies run to run on this GPU and one escaped run cannot on its own carry the constraint-scale claim. What carries it is that scale 0.0 escaped in three runs out of three (G7 seed 2700, G7b seed 2701, G10's own pre-ramp window) and that G10's collapse is locked to its ramp rather than to an epoch number.

What is NOT established. Which of the constraint terms does it. The scale multiplies mass balance, box, second law, budget, dissipation, parsimony and the thermodynamic prior together, and G9 (job 1677, `flux_off` at scale 0.1, no capacity, no budget, no thermodynamics) is the cell that starts to separate them; it reads spread 0.0000 at epoch 56, which points at the balance and box terms rather than the module's own, but one cell is not a decomposition.

7.12 Final wave-5 table ✗

Every number from `analyze_wave5.py` into `results/wave5_summary.json`. Escape is the first epoch beginning three consecutive epochs above a 0.05 validation prediction spread ratio. The endpoint is the pinned test Spearman on the 629 Merzbacher genes at the validation-selected epoch.

cell	job	arm	head	cstr scale	escape	test Spearman
G8	1674	pooled	n/a	n/a	26	+0.0717
G8b	1680	pooled	n/a	n/a	27	+0.1058
G7	1673	flux_capacity	fixed	0.0	33	+0.0747
G7b	1679	flux_capacity	fixed	0.0	26	+0.1185
G10	1678	flux_capacity	fixed	0 then 0.1 from ep 40	41	+0.0453
G6	1669	flux_capacity	fixed	0.1	none	-0.0160
G1-kj	1667	flux_capacity	unfixed	0.1	none	+0.0201
G9	1677	flux_off	fixed	0.1	none in 75	partial

Baselines: Flux Cone Learning +0.0391; the ten 020_v4 CGT cells +0.0779 with an across-seed SD of 0.0492.

The module against its control, paired by seed. Module +0.0747 and +0.1185, control +0.0717 and +0.1058. The paired differences are +0.0030 and +0.0127, mean **+0.0078**. Two pairs cannot support an interval, and the honest reading is that this is consistent with zero and with a small positive effect alike. One thing the pairing does show: the across-seed spread WITHIN an arm is 0.044, matching the 019 estimate, while the paired difference varies by 0.0069, so the two arms move together across seeds and the paired contrast is the right statistic for any future replication. An unpaired comparison at this n would be hopeless.

G10's test score is not the collapsed model's. Its selection picked epoch 45, inside the escaped window, and the checkpoint ring restored that snapshot before the test pass, so +0.0453 measures the escaped head at epoch 45 rather than the constant predictor the run ended as. A run that collapses after its selected epoch is scored correctly by this protocol, which is worth knowing since it is the case the ring was not designed for.

Where wave 5 leaves the question. The metabolic module can now be trained to the control's level, which no cell before today managed, and at that level it shows no measurable advantage over the control at two seeds. Its constraint terms as currently formulated cannot be applied at any weight above about 0.045 without returning the head to the mean plateau, before or after it has learned. The next thing to measure is which of the seven terms in that sum does it, since `flux_off` at scale 0.1 (G9) is also flat and carries none of the module's own.

7.13 G9 localizes the blocking terms: it is mass balance and the box, not the module ✗

G9 (job 1677) is `flux_off` with the head fix at constraint scale 0.1: `thermo_mode` OFF, `use_enzyme_capacity` False, `use_protein_budget` False, `use_dissipation_limit` False. Its constraint sum therefore contains only mass balance (weight 0.05), the box (0.5) and parsimony (1e-3); capacity, budget, thermodynamics, dissipation and the thermodynamic prior are all identically zero.

It never escaped in 80 epochs, ending at spread 0.0000 with a pinned test Spearman of **-0.0168**, indistinguishable from G6's -0.0160 which carried the full constraint set. Its constraint term ran 0.0118 against a data term of 0.530, 2.2 percent.

So the terms that block are the ones every flux arm shares, mass balance and the flux box, and none of the module's own physics is needed to produce the failure. That narrows the next experiment from seven terms to two, and it means the finding is about the differentiable constraint formulation in general rather than about enzyme capacity, the protein budget or the thermodynamics specifically. It also explains why arms as different as `flux_off`, `flux_capacity` and `flux_abundance` all behaved identically for five waves.

Final endpoint on the 629 pinned genes, every cell at 80 epochs: pooled +0.0717 and +0.1058; flux_capacity with constraints off +0.0747 and +0.1185; flux_capacity with constraints delayed to epoch 40 +0.0453; flux_capacity at scale 0.1 -0.0160; flux_off at scale 0.1 -0.0168; flux_capacity at scale 0.1 without the head fix +0.0201.